Conceptual Exploration of Equational Theories of Magmas

Implementation notes, case studies, and a user's guide

The `conceptual-exploration` library

October 2026

Abstract

This document describes how the `conceptual-exploration` library carries out the attribute exploration of implications between equational laws of magmas, the question studied by the Equational Theories Project. It is written for a reader who knows attribute exploration as presented in Ganter and Obiedkov's *Conceptual Exploration*, and it concentrates on what is done differently from the book: partial objects, symmetries, experts that cannot always decide, and the resumption of an exploration when an implication accepted without proof is refuted. It then describes what is specific to equational theories: the attributes and objects, background knowledge derived by bounded equational reasoning, and three experts: one searching for finite counterexamples with the Z3 SMT solver, one using the Vampire theorem prover, which also refutes implications with free magmas, and one deciding questions by Knuth–Bendix completion. Six case studies are discussed in detail: sixteen classical laws, all thirty-one one-variable laws with at most four operations, the ETP laws of order at most two and three, and the semigroup laws of order at most four, whose implications from several laws have been determined, or almost, by other means in other work, and the commutative and all semigroup laws of order at most five, beyond the ETP's laws, where some counterexamples, too large for any model search, are constructed instead. A user's guide closes the document.

Contents

1 Introduction **2**

2 Attribute exploration as implemented **3**
2.1 Overview . . . 3
2.2 Partial objects and the question asked . . . 3
2.3 Background knowledge . . . 5
2.4 Symmetries . . . 7
2.5 Experts that cannot always decide . . . 7
2.6 Resuming after a refuted unconfirmed implication . . . 9
2.7 Further details . . . 9

3 Exploring equational theories **10**
3.1 Attributes . . . 10
3.2 Objects: finite and free magmas . . . 10
3.3 Duality . . . 11
3.4 Background implications from bounded equational reasoning . . . 11
3.5 Background implications from the ETP . . . 13
3.6 The Z3 expert . . . 15
3.7 The Vampire expert . . . 15
3.8 Free magmas . . . 17

3.9 The completion expert . . . 18
3.10 Seeding with known magmas . . . 18

4 Case study: sixteen classical laws 18
4.1 Two explorations . . . 18
4.2 Where the experts differ . . . 18

5 Case study: one-variable laws of order at most four 20
5.1 Background . . . 20
5.2 Results . . . 20

6 Case study: ETP laws of order at most two and three 22
6.1 Choice of the laws . . . 23
6.2 Order at most two . . . 23
6.3 Order at most three . . . 23

7 Case study: semigroup laws of order at most four 27
7.1 Setup . . . 27
7.2 Results . . . 27
7.3 How the model search was extended . . . 29

8 Case study: commutative semigroup laws of order at most five 30
8.1 Setup . . . 30
8.2 Countermodels too large to search for . . . 30
8.3 Results . . . 31

9 Case study: semigroup laws of order at most five 32
9.1 Classes of laws . . . 32
9.2 Setup . . . 32
9.3 Truncated free semigroups . . . 33
9.4 Constructed seeds . . . 33
9.5 Results . . . 34
9.6 Certificates . . . 34
9.7 Lean . . . 36

10 User's guide 36
10.1 Installation . . . 36
10.2 Describing the laws . . . 37
10.3 Running an exploration . . . 38
10.4 Choosing the expert and the time limits . . . 38
10.5 Background knowledge . . . 38
10.6 Reusing magmas . . . 39
10.7 Reading the output . . . 39
10.8 Using the library from Python . . . 40
10.9 Practical remarks . . . 41

1 Introduction

A *magma* is a set G with a binary operation $*$. An *equational law* (or *law*) is an equation $s = t$ between two terms built from variables with $*$, read as universally quantified: G *satisfies* $s = t$, written $G \models s = t$, if the two sides agree under every assignment of elements of G to the variables. Examples are commutativity $x * y = y * x$, associativity $(x * y) * z = x * (y * z)$ and

idempotence $x = x * x$. The *order* of a law is the number of occurrences of $*$ in its two sides together.

The Equational Theories Project (ETP) [ETP25], initiated by Terence Tao in 2024, set out to decide, for every pair of laws E, F from a list of 4694 laws of order at most four, whether every magma satisfying E also satisfies F. Its laws are numbered, and we keep its numbers: Eq43 is commutativity, Eq4512 associativity, and so on.

From the point of view of Formal Concept Analysis, the natural object of study is the formal context

$$\mathbb{K}_{\mathrm{Mag}} = (\mathcal{G}, M, \models),$$

where M is a finite set of laws and $\mathcal{G}$ is the class of all magmas. Its object class is not even a set, but its implications are perfectly well defined: $P \to C$ holds in $\mathbb{K}_{\mathrm{Mag}}$ if every magma satisfying all laws of P satisfies all laws of C. By Birkhoff's completeness theorem, this is the case exactly when every law of C can be derived from the laws of P by the rules of equational logic. Its intents are the sets $\mathrm{Th}(G) \cap M$ for magmas G, the traces on M of equational theories. Attribute exploration is the natural tool for computing the canonical basis of $\mathbb{K}_{\mathrm{Mag}}$ and, with it, all its intents: an expert either confirms an implication, ideally by a proof, or refutes it by a magma.

The library implements the exploration algorithm of the book [GO16] in a general form, in the Python package `conceptual_exploration`, and the equational-theories domain in the directory `explorations/equational_theories`. Section 2 describes how the general algorithm differs from the book. Section 3 describes the domain: attributes, objects, the duality symmetry, background knowledge, and the experts. Sections 4, 5, 6, 7, 8 and 9 discuss six explorations, and Section 10 explains how to run one's own.

2 Attribute exploration as implemented

2.1 Overview

The exploration loop is the one of the book: the premises are the sets closed under the implications known so far, enumerated with NextClosure in lectic order, and for each premise P the expert is asked about $P \to C$, where C is what the current context suggests P implies. Four features change what happens around this loop:

1. objects are *partial*: for some attributes it may be unknown whether an object has them (Section 2.2);
2. a group of *symmetries* acts on the attributes, and implications and objects are added together with their images (Section 2.4);
3. the expert may *fail to decide* a question, in which case parts of the conclusion may be accepted without proof or rejected (Section 2.5);
4. when an implication accepted without proof is later refuted, the exploration is *resumed* at the point where it went wrong (Section 2.6).

Algorithm 1 summarizes the loop; the notation is explained in the following subsections.

2.2 Partial objects and the question asked

An object g of the context is a *partial object*: a pair (g^+, g^-) of disjoint sets of attributes, those g is known to have and those it is known not to have. The attributes in neither set are unknown. An object refutes $P \to C$ if $P \subseteq g^+$ and $C \cap g^- \neq \emptyset$.

The conclusion offered to the expert is computed with the closure

$$P^{\diamond} = M \setminus \bigcup \{\, g^- : g \text{ an object with } P \subseteq g^+ \,\}.$$

Algorithm 1 The exploration loop (`AttributeExploration.run`)

Require: attributes M with a linear order, background implications $\mathcal{B}$, symmetries Φ, expert

$\mathcal{L} \leftarrow \mathcal{B}$ closed under Φ; $\quad \mathcal{T} \leftarrow \mathcal{L}$ ▷ all implications; trusted ones
$P \leftarrow \mathcal{L}(\emptyset)$
loop
  $C \leftarrow P^{\diamond} \setminus P$ ▷ context closure, Section 2.2
  if $C = \emptyset$ **then**
    $P \leftarrow$ next $\mathcal{L}$-closed set after P; **continue**
  end if
  if $P \to C$ is known false **then** ▷ Section 2.5
    add a derived witness for each $c \in C$ with $P \to c$ known false; if there is one, **continue** with the same P
    $C \leftarrow \{c\}$ for the first $c \in C$
  end if
  ask the expert about $P \to C$
  if the answer is an object g refuting $P \to C$ **then**
    add g and its images under Φ, completed with $\mathcal{T}$; **continue** with the same P
  else if the answer is "accept" **then**
    add $P \to C$ and its images to $\mathcal{L}$ (and to $\mathcal{T}$ if the expert decided it)
  else ▷ undecided: proved part $D \subseteq C$, partial object, flag
    add the partial object; add $P \to D$ to $\mathcal{L}$ and $\mathcal{T}$; $R \leftarrow C \setminus \mathcal{T}(P \cup D)$
    add $P \to R$ to $\mathcal{L}$ only (unconfirmed), or reject it
  end if
  if an object or a rejected implication refutes an implication of $\mathcal{L} \setminus \mathcal{T}$ **then**
    stop and resume (Section 2.6)
  end if
  $P \leftarrow$ next $\mathcal{L}$-closed set after P (or stop if there is none)
end loop

An attribute belongs to $P^{\diamond}$ unless some object known to have all of P is known not to have it; $P^{\diamond}$ is thus the largest conclusion not refuted by the data at hand. When every row is complete, $P^{\diamond} = P''$ and the question is the one of the book. It is the cautious choice for incomplete data, and it is what makes partial rows useful: an object whose row is only partly known still removes from later conclusions the attributes it is known to lack.

In the equational-theories exploration, finite magmas always have complete rows (the expert evaluates every law on every counterexample), while free magmas (Section 3.8) may have unknown entries.

Completion. Every object is *completed* with the *trusted* implications $\mathcal{T}$ (background, confirmed, and their images; see Section 2.5): its positive side is replaced by its $\mathcal{T}$-closure, and every unknown attribute m whose addition would force an attribute of g^- is added to the negative side:

$$g^+ := \mathcal{T}(g^+), \qquad g^- := g^- \cup \{\, m : \mathcal{T}(g^+ \cup \{m\}) \cap g^- \neq \emptyset \,\},$$

the second step repeated until nothing changes. If the closure of the positive side meets the negative side, the object contradicts a trusted implication, and the program stops with an error: such an object can only come from a faulty expert. Objects are completed when they are added, and all objects are brought up to date whenever new trusted implications are accepted. Completion is what eventually fills in some unknown entries of partial rows.

2.3 Background knowledge

Background implications are given before the exploration starts, as in the book. They take part in the closure of premises and in the completion of objects, and they are reported with the source `background`. Without the features of the next subsections, the accepted implications form the canonical basis of the context relative to the background implications.

The rest of this subsection describes a general way to obtain background implications from known implications between a larger set of attributes, possibly through attributes that are not explored. Section 3.4 applies it to laws, with the instances of the inference rules of equational reasoning as the known implications.

Known implications. Let the explored attributes M be part of a larger, possibly infinite, set U of attributes, all of which make sense for every object of the domain. For $P \subseteq U$ and $u \in U$, write $P \models u$ if every object of the domain that has all attributes of P has u. Let $\mathcal{R}$ be a set of implications $A \to u$, each with a finite premise $A \subseteq U$ and a single conclusion $u \in U$, that hold in the domain: $A \models u$. For $X \subseteq U$, let $\mathcal{R}|X$ be the set of the implications of $\mathcal{R}$ whose premises and conclusion all lie in X, and for $P \subseteq X$, let $\mathrm{cl}_X(P)$ be the least subset of X that contains P and the conclusion of every implication of $\mathcal{R}|X$ whose premises it contains. Equivalently, $\mathrm{cl}_X(P)$ consists of the attributes that have a derivation from P, a finite sequence of attributes each of which is in P or the conclusion of an implication of $\mathcal{R}|X$ whose premises precede it. The implication set $\mathcal{R}$ is *complete* if $\mathrm{cl}_U(P)$ contains every u with $P \models u$, for every finite $P \subseteq U$.

Definition 1. For $M \subseteq X \subseteq U$ and $P \subseteq M$, let $\sigma_X(P) = \mathrm{cl}_X(P) \cap M$.

Proposition 1. *Let $M \subseteq X \subseteq Y \subseteq U$.*

1. *σ_X is a closure operator on M, whose closed sets are the sets $C \cap M$ for the sets $C \subseteq X$ closed under $\mathcal{R}|X$.*
2. *For every $P \subseteq M$, $\sigma_X(P) \subseteq \sigma_Y(P) \subseteq \{\, m \in M : P \models m \,\}$.*
3. *If $\mathcal{R}$ is complete and U is the union of a chain $X_1 \subseteq X_2 \subseteq \cdots$ of sets containing M, then for every finite $P \subseteq M$ and every $m \in M$ with $P \models m$, $m \in \sigma_{X_k}(P)$ for some k.*

Proof. (1) Since $P \subseteq M \subseteq X$, σ_X is extensive, and it is monotone since cl_X is. It is idempotent: $\sigma_X(\sigma_X(P)) \subseteq \mathrm{cl}_X(\mathrm{cl}_X(P)) \cap M = \sigma_X(P)$. A σ_X-closed set Q is $\mathrm{cl}_X(Q) \cap M$; conversely, if C is closed under $\mathcal{R}|X$, then $\sigma_X(C \cap M) \subseteq \mathrm{cl}_X(C) \cap M = C \cap M$. (2) Every implication of $\mathcal{R}|X$ is in $\mathcal{R}|Y$, so $\mathrm{cl}_Y(P) \cap X$ is closed under $\mathcal{R}|X$ and contains $\mathrm{cl}_X(P)$. The second inclusion holds since the implications of $\mathcal{R}$ hold in the domain. (3) By completeness, m has a derivation from P with implications of $\mathcal{R}$. It involves finitely many attributes, which all lie in some X_k, and then all its implications are in $\mathcal{R}|X_k$. □

Every implication $P \to \sigma_X(P)$ thus holds in the domain, and a larger X gives more of them, at the cost of computing with $\mathcal{R}|X$. In the terms of formal concept analysis, σ_X is the closure operator of the implications of the context whose intents are the sets closed under $\mathcal{R}|X$, restricted to the attributes in M; it entails more than the implications of $\mathcal{R}|M$ alone, since derivations may pass through attributes of $X \setminus M$.

Transient attributes. Let $M \subseteq N \subseteq T \subseteq U$, where $\mathcal{R}|N$ can be computed in advance but $\mathcal{R}|T$ is too large to be. Some derivations through $T \setminus N$ can still be followed, starting from what has been derived in N.

Definition 2. A *transient path* from a set $D \subseteq N$ is a sequence $t_1, \ldots, t_k \in T \setminus N$, $k \geq 1$, followed by $t \in N$, together with implications $A_0 \to t_1$, $A_1 \cup \{t_1\} \to t_2$, …, $A_k \cup \{t_k\} \to t$

of $\mathcal{R}|T$, where $A_0, \ldots, A_k \subseteq D$. Its *result* is t, and it gives the implication $A_0 \cup \cdots \cup A_k \to t$ over N. Let $\Pi_{N,T}$ be the set of the implications given by transient paths, and, for $P \subseteq N$, let $\mathrm{cl}_{N,T}(P)$ be the least subset of N that contains P and is closed under $\mathcal{R}|N \cup \Pi_{N,T}$. For $P \subseteq M$, let $\sigma_{N,T}(P) = \mathrm{cl}_{N,T}(P) \cap M$.

A derivation through a transient path leaves N once, uses each attribute outside N once, as the premise of the next step, and takes all other premises from N. The set $\mathrm{cl}_{N,T}(P)$ is the least subset of N containing P that is closed under $\mathcal{R}|N$ and contains the result of every transient path from itself; it can therefore be computed by alternating closure under $\mathcal{R}|N$ with a search for transient paths from what has been derived, without $\mathcal{R}|T$ being computed. The search may be restricted, for instance in the implications allowed at each step:

Proposition 2. *Let $M \subseteq N \subseteq T \subseteq U$, let Π be a set of implications over N each of which has a derivation from its premise with implications of $\mathcal{R}|T$ (such as any $\Pi \subseteq \Pi_{N,T}$), and let $\sigma'(P)$ be the closure of $P \subseteq M$ under $\mathcal{R}|N \cup \Pi$, intersected with M. Then σ' is a closure operator on M and*

$$\sigma_N(P) \;\subseteq\; \sigma'(P) \;\subseteq\; \sigma_T(P) \qquad \text{for every } P \subseteq M.$$

In particular, $\sigma_N \subseteq \sigma_{N,T} \subseteq \sigma_T$, and $\sigma_{N,N} = \sigma_N$.

Proof. σ' is a closure operator as in Proposition 1, and the first inclusion holds since $\mathcal{R}|N \subseteq \mathcal{R}|N \cup \Pi$. For the second, $\mathrm{cl}_T(P) \cap N$ is closed under $\mathcal{R}|N$, and under every implication $A \to t$ of Π: if it contains A, then $\mathrm{cl}_T(P)$ contains the derivation of t from A, and $t \in N$. For $N = T$ there is no transient path. □

The second inclusion is strict in general: a derivation with $\mathcal{R}|T$ may leave N several times, combine two attributes outside N in one implication, or use one twice, and a transient path does none of these. Transient attributes thus trade completeness for cost: σ_T requires the implications of $\mathcal{R}|T$, whereas transient paths start only from attributes already derived and keep only those of N.

From a closure operator to background implications. Let σ be any of these closure operators on M; every implication $P \to \sigma(P)$ holds in the domain. The background implications are computed from σ in one of two ways:

- With a *premise size* $K \geq 1$, the background B_K starts with $\emptyset \to \sigma(\emptyset)$, and the sets $P \subseteq M \setminus \sigma(\emptyset)$ with $|P| \leq K$ are considered by increasing size, $P \to \sigma(P) \setminus P$ being added unless the implications added so far entail it. The Horn closure under B_K coincides with σ on sets of at most K attributes, and is contained in σ on larger sets, possibly strictly: attributes that more than K attributes give only jointly are missed.
- With $K = 0$, the background B_0 is the canonical basis of σ, computed with NextClosure. The Horn closure under B_0 is σ, but NextClosure visits every σ-closed set.

Corollary 3. *Let $M \subseteq N \subseteq T \subseteq U$ and $K \geq 0$. Every implication of the background B_K computed from $\sigma_{N,T}$, or from any σ' as in Proposition 2, is entailed by the background B_K computed from σ_T. If the former entails every implication between attributes of M that holds in the domain, the two backgrounds are therefore equivalent.*

Proof. An implication $P \to Q$ of the first background satisfies $Q \subseteq \sigma'(P) \subseteq \sigma_T(P)$, and $|P| \leq K$ unless $K = 0$. In either case, the Horn closure of P under the second background contains $\sigma_T(P) \supseteq Q$. The second claim follows since both backgrounds hold in the domain. □

2.4 Symmetries

A *symmetry* ("mapping" in the code) is a map φ from the attributes to the attributes. When an implication $P \to C$ is accepted, so is its image $\varphi(P) \to \varphi(C)$ for every $\varphi \in \Phi$, unless the theory already entails it; images are reported with the source `mapped`, or `mapped unconfirmed` for images of unconfirmed implications. When an object g is added, so is, for every $\varphi \in \Phi$, a copy of g with the row

$$(\varphi^{-1}(g^+),\ \varphi^{-1}(g^-)),$$

the object that has an attribute m exactly when g has $\varphi(m)$. Copies are named $g\#k$, where k is the number of the mapping (the original is $g\#0$), and a copy whose row repeats one already added is skipped. Each implication and each object is mapped by each φ once and never by a composition, so Φ should be closed under composition, for instance a group. (In general a map may also send an attribute to a truth value; this is used in rule exploration, not here.)

Since the theory is closed under Φ at all times, the premise $\varphi(P)$ of the image of an accepted implication is not closed and never becomes a question: the expert is asked about one implication per orbit. The images of an unconfirmed implication are no more certain than it, but they steer the enumeration as it does. With `map_unconfirmed=False` (`--ask-unconfirmed-duals` in `explore.py`), they are added only once the implication is confirmed; until then the expert is asked about them when the enumeration reaches their premises, and if it proves an image, the implication is recorded as confirmed as well. The price is that the resulting set of implications, background, accepted and mapped together, is only a *cover* of the theory: a set of implications that generates it, possibly with redundant implications, and in general not its canonical basis.

2.5 Experts that cannot always decide

The book assumes an expert who always answers correctly. The experts used here are programs with time limits, and a question may be left undecided. The library distinguishes five sources of implications:

`background` given in advance;

`confirmed` accepted by an expert who decided the question;

`unconfirmed` accepted without a decision: the expert found no counterexample but cannot rule one out;

`mapped` the image of a background or confirmed implication under a symmetry;

`mapped unconfirmed` the image of an unconfirmed implication, no more certain than it; it becomes `mapped` if the implication is confirmed later.

An expert reports that it has not decided a question it accepts through `is_conclusive()`. In addition, an expert may answer with an `Undecided` value, which carries

- a set $D \subseteq C$ of conclusion attributes it did prove: $P \to D$ is accepted as confirmed;
- optionally a partial object, which must not refute $P \to C$ and is added to the context (its unknown entries may be filled later by completion);
- a flag, `rejected`, telling what to do with the rest $P \to R$, where R consists of the attributes of C that do not follow from $P \cup D$ by the trusted implications (those that do are proved with D): it is accepted as *unconfirmed* if the expert knows nothing against it (or, in the other mode below, rejected tentatively), or *rejected* if the expert knows it to be false without knowing which attribute fails.

An expert that knows a single attribute c to fail returns a counterexample instead, if need be the partial object with the row $(P, \{c\})$. The partial object of an `Undecided` answer, with

what was proved, may show the rest false, as defined next, although the question was not. The attributes of the rest that an object then refutes, or that are known false on their own, are settled as before a question; the others, which the expert knows nothing against, are accepted as unconfirmed, together if they contradict nothing together and one by one otherwise. Nothing is asked again. Besides its answers, an expert may give objects it has come to know along the way, such as the other candidates of a search, through `new_objects()`; they are added to the context once the answer has been applied, and an object given again merges into the one given before.

Trusted and untrusted implications. Unconfirmed implications take part in the closure that generates the premises: they steer the enumeration as accepted implications do. They are, however, not used to complete objects, since they may be false and would then write false values into rows. Each unconfirmed implication, together with its images if they are added and with the trusted implications, must not contradict an object, that is, give it an attribute it lacks, nor entail a rejected implication; an object or a rejected implication it contradicts refutes it. This is checked whenever an object, a trusted implication or a rejected implication comes in, since a trusted implication may make an unconfirmed one contradict a partial object it did not contradict before. Unconfirmed implications that contradict an object or a rejected implication only together are allowed: one of them is false, but which is not known. They are reported as *joint conflicts*: for each partial object or rejected implication that all implications contradict together, the unconfirmed implications are dropped one at a time, in the order they were accepted, as long as the others still contradict it, which leaves a minimal set. (A complete object contradicted by the implications together is refuted by one of them, since it is closed under the trusted implications.)

Rejecting tentatively. Accepting an undecided implication as unconfirmed lets it steer the enumeration: no question is asked whose premise it would close further. The other mode (`undecided="reject"`) takes it to be false instead, which asks more questions but steers by nothing that is not known. For each attribute r of the undecided rest R, a *tentative object* with the row $(P, \{r\})$, completed with $\mathcal{T}$, joins the context. It takes part in the context closure, so that r leaves the conclusions of the premises it has, but in nothing that needs facts: it is not mapped by the symmetries, shows no question false, refutes no unconfirmed implication, and is left out of the saved contexts; the implications it stands for are listed as open. No unconfirmed implication is accepted in this mode. If the trusted implications come to entail $P \to r$, the tentative object is refuted. Without symmetries this cannot happen, since the enumeration is past P when the implication is added; an image of a later implication may entail it, though.

Questions known false. A question $P \to C$ is *known false* if, together with its images under the symmetries and with the trusted implications, it contradicts an object or entails a rejected implication. The images count whether or not they are added to the theory, since an implication with a false image is false itself. A question known false is not asked. Each attribute $c \in C$ for which $P \to c$ is known false gets a *derived witness* with the row $(P, \{c\})$, which takes c out of $P^\diamond$. If there is none, the attributes of C contradict only together; this happens when completion leaves them unknown in a partial object, as with an object lacking d, unknown on b_1 and b_2, and a trusted $b_1 b_2 \to d$. The expert is then asked about $P \to c$ for the first $c \in C$ alone. No question entailing a preimage of a rejected implication is asked either: mapping such a question with its images and the trusted implications shows that they entail the rejected implication itself. A confirmed implication therefore never contradicts the objects or the rejected implications.

Rejected implications. A rejected implication is not accepted, and the enumeration continues with the next closed set as if the context had no question about P. The theory remains sound. It may, however, be incomplete at P: the expert knows that $P \to R$ fails as a whole, but some of the attributes of R may still follow from P. Rejected implications are listed separately in the results. If the rest has a single attribute c, though, it is not rejected: some object has the attributes of P, and those of D, which P implies, and lacks c, and nothing else is known about it. Such a *witness* is a counterexample, if a partial one, and it joins the context with the row $(P \cup D, \{c\})$; in the results, witnesses are named `Wk`.

A rejected implication is a fact, which no row can express: it says that some attribute of R fails, but not which. It is used as a constraint instead: questions entailing it are known false, and an unconfirmed implication entailing it with the trusted implications is refuted. A rejected implication is never entailed when it is recorded, since its premise is closed under the theory.

2.6 Resuming after a refuted unconfirmed implication

When an object or a rejected implication refutes an unconfirmed implication $P \to C$, all closed sets enumerated after P were computed with a false implication, and the enumeration from P on cannot be trusted. If several are refuted at once, $P \to C$ is the one accepted first. A refuted tentative object, added at the question about P, is treated the same way, and so are the tentative objects added after it. The exploration then stops and is resumed (`explore_in_rounds`): a new round starts with

- the implications accepted before $P \to C$, whatever their source;
- the *confirmed* implications accepted after it, which are true whatever steered the enumeration to them;
- every object found so far, and every rejected implication, which is false whatever steered the enumeration to it;

and resumes NextClosure at P. Since NextClosure can continue from any set, and every closed set lectically before P was enumerated without the false implication, nothing before P is repeated. The false implication and the unconfirmed implications accepted after it are dropped; their questions are asked again if they arise. The implications found are then a cover of the theory, sound and complete up to rejected implications, but, because the confirmed implications kept were found under a different theory, not necessarily its canonical basis.

2.7 Further details

- **Several questions per premise.** After a counterexample, the context closure of the same premise is recomputed and, if it is still larger than the premise, the expert is asked again, as in the book.
- **Continuing after an acceptance.** After $P \to C$ is accepted, the next closed set is computed from P under the extended theory. NextClosure reuses the step that produced P: the closure of P under the new theory is the next closed set if it adds no attribute lectically smaller than the one just added.
- **Rows of counterexamples.** With `evaluate_all`, the expert is given all attributes and returns the full row of a counterexample, not only its values on the question's attributes. The equational-theories exploration always does this.
- **Bit sets.** Attribute sets, rows and implications are stored as bit masks, and distinct rows are stored once with the objects sharing them; the context closure and the implication closure (a variant of Wild's algorithm) are integer operations.

- **Reading the result.** For display, `simplified_implications` simplifies the accepted implications, using only the *trusted* implications: the background, the confirmed implications and their images. Each premise is cut to the attributes the rest of it does not entail by them. The conclusion of a simplified confirmed implication starts as everything its premise entails by them, that of a simplified unconfirmed implication as what the implication adds to this, and each conclusion is cut to the attributes that do not follow from the premise by way of the background, the other simplified confirmed implications and all their images. A simplified implication left with no conclusion follows from these and is dropped. The simplified confirmed implications are thus reduced, none of them following from the others together with the background and the symmetries, and they form a basis of the trusted implications relative to the background and the symmetries, though not in general its canonical basis, nor one with as few implications as possible. A simplified unconfirmed implication is never cut by another unconfirmed one, so that each states one step that is not proved: no attribute of its conclusion follows from its premise by the trusted implications, and two open steps are not merged into one; it may, however, follow from other unconfirmed ones. A premise that entails every attribute is shown as concluding *everything*. The simplified implications, read together with the background and the symmetries, entail exactly what the accepted implications do.

3 Exploring equational theories

3.1 Attributes

The attributes are the laws listed in a JSON file, each with an equation string and optionally a name and an ETP number:

```
[
  {"id": 43, "name": "Commutativity", "equation": "(x * y) = (y * x)"},
  {"id": 4512, "name": "Associativity", "equation": "((x * y) * z) = (x * (y * z))"},
  {"name": "Medial / Entropic", "equation": "((x * y) * (z * w)) = ((x * z) * (y * w))"}
]
```

Terms are parsed into trees with variables as leaves and $*$ as the only operation; laws are compared syntactically. The set of laws must be closed under duality (Section 3.3).

3.2 Objects: finite and free magmas

A *finite magma* is given by its Cayley table on $\{0, \ldots, n-1\}$. Whether it satisfies a law is decided by evaluating both sides under all n^k assignments to the k variables of the law, so rows of finite magmas are always complete. Most laws that fail do so on one of a few random assignments, which are tried first; the others are evaluated on blocks of assignments at once. When the magma has a zero, an element z with $z * a = a * z = z$ for all a, that every term with at least k leaves takes, a side with k or more leaves is z whatever the assignment: a law with two such sides holds without any evaluation, and a law with one reduces to the other side being z. Whether this is so is decided by the values of the terms with k to $2k-2$ leaves, since a term with more leaves has a subterm with between k and $2k-2$ of them.

A *free magma* (Section 3.8) is a genuine magma, but usually an infinite one, so the library does not construct it; it represents it by its premise and computes only its row, which may be partial.

3.3 Duality

The *dual* of a term reverses all products, $(s * t)^{\mathrm{op}} = t^{\mathrm{op}} * s^{\mathrm{op}}$, and the dual of a law $s = t$ is $s^{\mathrm{op}} = t^{\mathrm{op}}$. The dual G^{op} of a magma has the transposed table, $a *^{\mathrm{op}} b = b * a$, and $G^{\mathrm{op}} \models E^{\mathrm{op}}$ if and only if $G \models E$. The map $E \mapsto E^{\mathrm{op}}$, with laws identified up to renaming variables and swapping sides, is the only symmetry used; it is an involution, so $\Phi = \{\,^{\mathrm{op}}\}$ is closed under composition. Counterexamples are thus added together with their duals, named `M`*k* and `M`*k*`^op` in the results, and implications together with their dual implications.

When every object satisfies associativity (option `--axiom`, which adds the empty-premise implication concluding the given laws to the background and drops seeded magmas that violate them), the objects are semigroups, and a law is determined by the two words obtained by dropping the parentheses. Laws are then compared as words, so that the dual of a law is the law with both words reversed, again up to renaming variables and swapping sides; the dual of a semigroup is a semigroup. Option `--no-duality` explores without the symmetry, for sets of laws that are not closed under it.

3.4 Background implications from bounded equational reasoning

For laws in a single variable, background implications can be derived syntactically, by equational reasoning restricted to equations of bounded size. This is an instance of the scheme of Section 2.3: the attributes U are the equations in one variable, the domain is that of all magmas, the known implications $\mathcal{R}$ are the instances of the inference rules of equational reasoning, and the sets $M \subseteq N \subseteq T$ are the explored laws, the equations of order at most n, and those of order at most n'.

Terms and equations. Let Tm be the set of terms built from a single variable x with the binary operation $*$, and let $|t|$ be the number of operations in a term t. An *equation* is an unordered pair of terms, written $s \approx t$; it is *trivial* if $s = t$, and its *order* is $|s| + |t|$. A law in the variable x is thus an equation, the order of its sides being immaterial. We write $\mathcal{E}_n$ for the finite set of equations of order at most n, $C[\,\cdot\,]$ for a context, that is, a term with one hole, and $t[u/x]$ for the term obtained from t by substituting u for every occurrence of x. For a set P of equations, $P \models e$ means that every magma satisfying the laws of P satisfies e.

Inference rules. Derivations use four inference rules, for all $s, t, u, a, b \in \mathrm{Tm}$:

(E) *reflexivity*: infer $t \approx t$ from no premise;

(M) *multiplication*: from $s \approx t$ infer $u * s \approx u * t$ and $s * u \approx t * u$;

(S) *substitution*: from $s \approx t$ infer $s[u/x] \approx t[u/x]$;

(R) *replacement*: from $a \approx b$ and an equation e infer the equation obtained from e by replacing one occurrence of a, in either side, by b.

Since equations are unordered, (R) may replace either side of $a \approx b$ by the other, and it subsumes transitivity: from $a \approx b$ and $a \approx c$ it infers $b \approx c$. An *instance* of an inference rule, given by its premises and its conclusion, is an implication in the sense of Section 2.3. All four inference rules are sound: a magma that satisfies the premises of an instance satisfies its conclusion. Let $\mathcal{R}$ be the set of their instances.

Definition 3. For $n \geq 0$ and $P \subseteq \mathcal{E}_n$, let $D_n(P) = \mathrm{cl}_{\mathcal{E}_n}(P)$ be the set of equations *derivable from P within order n*: the least subset of $\mathcal{E}_n$ that contains P and the conclusion of every instance in $\mathcal{R}|\mathcal{E}_n$ whose premises it contains.

$D_n(P)$ contains every trivial equation of $\mathcal{E}_n$, by (E). The set $\mathcal{R}|\mathcal{E}_n$ is finite. The implementation computes once the set I_n of its instances of (M), (S) and (R) whose premises and conclusion

are non-trivial, as Horn clauses with one or two premises over the numbered equations of $\mathcal{E}_n$, and computes the non-trivial equations of $D_n(P)$ by forward chaining with I_n. Leaving out trivial equations loses nothing:

Lemma 4. *For $P \subseteq \mathcal{E}_n$, the non-trivial equations of $D_n(P)$ are exactly those derivable from the non-trivial equations of P with the instances in I_n.*

Proof. A trivial equation $u \approx u$ yields only trivial equations by (M) and (S), and, as the first premise of (R), only its second premise. As the second premise of (R), with a non-trivial first premise $a \approx b$, it yields $C[b] \approx C[a]$, where $u = C[a]$. The same equation follows from $a \approx b$ by one application of (M) for each level of the context C, through the equations $C'[a] \approx C'[b]$ for the subcontexts C' of C. These are non-trivial, since $a \neq b$, and their order does not exceed that of $C[a] \approx C[b]$. By induction on derivations, trivial equations, which only (E) and these steps produce, can therefore be eliminated without leaving $\mathcal{E}_n$. □

Proposition 5. *Let $P \subseteq \mathcal{E}_n$ and $n \leq n'$. Then*

$$D_n(P) \;\subseteq\; D_{n'}(P) \cap \mathcal{E}_n \;\subseteq\; \{\, e \in \mathcal{E}_n : P \models e \,\},$$

and every $e \in \mathcal{E}_n$ with $P \models e$ belongs to $D_{n'}(P)$ for some $n' \geq n$.

Proof. The inclusions are those of Proposition 1, with $M = X = \mathcal{E}_n$ and $Y = \mathcal{E}_{n'}$. The sets $\mathcal{E}_{n'}$ form a chain whose union is U, so by the same proposition it remains to show that $\mathcal{R}$ is complete. Let $P \models s \approx t$. By Birkhoff's completeness theorem [BN98, Chapter 3], there is a chain of terms $s = t_0, t_1, \ldots, t_k = t$ in which each t_i is obtained from t_{i-1} by replacing a subterm $l\theta$ by $r\theta$, for a law $l \approx r$ of P, used in either direction, and a substitution θ. Mapping every variable to x turns it into such a chain of terms of Tm, with substitutions $x \mapsto w_i$, since s and t contain no other variable. (E) gives $s \approx t_0$. For the i-th step, (S) gives $l[w_i/x] \approx r[w_i/x]$ from $l \approx r$, and (M), applied once for each level of the context of the replaced subterm, gives $t_{i-1} \approx t_i$. Then (R), replacing the side t_{i-1} of $s \approx t_{i-1}$ by t_i, gives $s \approx t_i$. □

The implication set $\mathcal{R}$ is thus complete for one-variable laws, but a derivation may have to pass through equations of much higher order than its premises and its conclusion, so that D_n is incomplete for every fixed n. Raising n is expensive: the number of terms with k operations is the Catalan number C_k, and $\mathcal{E}_n$ has 30 non-trivial equations for $n = 4$, 1,023 for $n = 7$ and 41,217 for $n = 10$, with correspondingly many instances.

Transient order. A second bound $n' \geq n$, the *transient order*, admits transient paths through $T \setminus N = \mathcal{E}_{n'} \setminus \mathcal{E}_n$ of one particular shape.

Definition 4. Let $D \subseteq \mathcal{E}_n$ be a set of non-trivial equations. For terms $u, v \in \mathrm{Tm}$, write $u \to_D v$ if v is obtained from u by replacing one occurrence of a subterm a by b, where $a \approx b \in D$ and $|b| \leq |a|$. A *transient step* from D takes an equation $e \in D$ and an instance of (M) or (S) with premise e whose conclusion $s' \approx t'$ has order greater than n and at most n', and yields every non-trivial equation $s'' \approx t'' \in \mathcal{E}_n$ with $s' \to_D^* s''$ and $t' \to_D^* t''$. For $P \subseteq \mathcal{E}_n$, $D_{n,n'}(P)$ is the least subset of $\mathcal{E}_n$ that contains $D_n(P)$ and is closed under $\mathcal{R}|\mathcal{E}_n$ and under transient steps from itself.

In particular, $D_{n,n} = D_n$. The equation $s' \approx t'$ and the equations between it and the result are not kept: they are never premises of later steps, nor used for rewriting. Since transient steps depend on what has been derived, they are not instantiated in advance; the implementation alternates forward chaining with transient steps from all equations derived so far, until neither adds an equation. It explores at most 5,000 terms from each side, which can only remove conclusions; the closure operators below disregard this bound. For example, substituting $x * x$ for x in $x * x \approx ((x * x) * x) * x$ gives an equation of order 10, which $x \approx (x * x) * (x * x)$ rewrites to $x \approx (x * (x * x)) * (x * x)$, of order 4.

Proposition 6. *For $P \subseteq \mathcal{E}_n$ and $n \leq n'$,*

$$D_n(P) \;\subseteq\; D_{n,n'}(P) \;\subseteq\; D_{n'}(P) \cap \mathcal{E}_n.$$

Proof. A transient step from D is a derivation of its result from D with instances in $\mathcal{R}|\mathcal{E}_{n'}$. The instance of (M) or (S) that gives $s' \approx t'$ from $e \in D$ is one. A rewrite $u \to_D v$ in a side of an equation $u \approx w$ is an instance of (R) with premises $a \approx b \in D$ and $u \approx w$ and conclusion $v \approx w$, whose order does not exceed that of $u \approx w$, since $|b| \leq |a|$; rewriting s' to s'' and then t' to t'' thus stays within $\mathcal{E}_{n'}$. Each transient step therefore gives an implication over $\mathcal{E}_n$ as in Proposition 2, with $N = \mathcal{E}_n$ and $T = \mathcal{E}_{n'}$, which gives the inclusions. □

Until it first returns to $\mathcal{E}_n$, a transient step is a transient path in the sense of Section 2.3: each of its equations of order above n is used once, as the premise of the next step, with an equation of D as the other premise. Once in $\mathcal{E}_n$, its rewrites, which do not increase the order, are instances of $\mathcal{R}|\mathcal{E}_n$. Transient steps are restricted further: they start with a single instance of (M) or (S), rewrite only with equations of D that do not enlarge a term, and never use an equation of order above n for rewriting.

Background implications. Let $L \subseteq \mathcal{E}_n$ be the set of explored laws, and for $P \subseteq L$, let

$$\sigma(P) = D_{n,n'}(P) \cap L,$$

a closure operator σ' as in Proposition 2, with $M = L$, $N = \mathcal{E}_n$, $T = \mathcal{E}_{n'}$, and the implications given by the transient steps as Π. In particular, $\sigma(\emptyset)$ is the set of trivial laws of L, which hold in every magma. By Propositions 5 and 6, every implication $P \to \sigma(P)$ holds in all magmas, and the background B_K is computed from σ as described there, with premise size $K = 2$ by default. By Corollary 3, every implication of the background computed with order n and transient order n' is entailed by the background computed with order n' alone, with the same K; if the former entails every implication between the laws that holds in all magmas, the two are equivalent. For the 31 laws of Section 5, the background with $n = 4$, $n' = 10$ and $K = 2$ entails every implication that holds in all magmas, and is thus equivalent to the background of order 10, whose computation would require the instances over all 41,217 equations of $\mathcal{E}_{10}$.

Laws in several variables are not supported: rule (S) would then have to substitute for each variable separately and to rename variables apart, and laws would have to be identified up to renaming.

3.5 Background implications from the ETP

The ETP has decided every implication between two of its 4694 laws, and publishes the outcomes as an implication graph[1]: a 4694×4694 array, row the premise and column the conclusion, run-length encoded. Together with the ETP's list of laws[2], it gives background implications for laws with any number of variables:

- every explored law is matched with an ETP law by a canonical form: its variables are renamed in the order of their first occurrence, for both orders of the two sides, and the smaller of the two strings is taken. Two laws have the same canonical form exactly when one becomes the other by renaming variables and swapping sides. A law whose `id` is not the ETP's number of the matched law stops the program with an error; laws the ETP does not list take no part;
- the laws that $x = x$ implies in the graph, which hold in every magma, form the conclusion of an implication with empty premise;

[1] `https://teorth.github.io/equational_theories/implications/graph.json`
[2] `https://github.com/teorth/equational_theories/blob/main/data/equations.txt`

- every other listed law E yields $E \to \{F : E \text{ implies } F\}$, leaving out the laws valid in every magma.

The exploration then only asks about premises of several laws, since every implication with a single premise between listed laws is entailed by the background. The background is reported with the source `background`, like derived implications, and both kinds can be used together.

In the terms of Section 2.3, let $\mathcal{R}$ consist of the graph's implications and, since the background is closed under duality, their duals, and let N be the ETP's laws together with the explored laws M. The background consists of the implications of $\mathcal{R}|M$, and the trace σ_N can give more, through chains of implications that pass through laws outside the exploration; the mark `new` below is computed with σ_N. Among magmas the two agree, since the graph is closed under composition and duality. Among semigroups, where the dual of a law is its reversal, they differ: nine proved implications printed for the semigroup laws of order at most four follow by σ_N from a single law and associativity (Section 7).

For each pair of laws, the graph records whether the implication is proved, or refuted, explicitly or by inference from other results (outcomes 3 and 7, and 2 and 6, of the ETP's explorer); at the commit used, every pair has one of these four outcomes. Whether each is backed by a proof in the ETP's Lean formalization is not checked here, so a result with this background is as trustworthy as the ETP. A result proved in Lean, as in Section 9.7, does not rest on the graph: the implications its Lean files prove entail the background implications, as `verify.py --lean` checks.

The graph also gives the implications that the ETP refutes, though not its counterexamples, some of which are infinite. With `--etp-refutations`, each refuted implication $E \to F$ between listed laws is added to the context as a partial object (Section 2.2): the ETP's counterexample satisfies E and every law that E implies, and violates F and every law that implies F, and nothing else is known about it. These objects are reduced together with the seeded magmas (Section 3.10), so that only refutations that no other object accounts for remain; they are named `R`k in the results. They matter when the ETP's counterexample is infinite, or finite but beyond the reach of the expert, since a question refuted only by such a magma would otherwise be accepted as unconfirmed.

The ETP data used in this document are those of the ETP repository at commit `1aec8a7`, whose `equations.txt` and `graph.json` are recorded with their checksums in `etp_version.json`. The script `fetch_etp.py` downloads them, and the ETP's files of finite magmas used in Section 6, which are recorded there as well, and checks each file against its checksum: `fetch_etp.py DIR` puts `equations.txt` and `graph.json` into `DIR`, and the files of magmas into `DIR/magmas`.

The script `etp.py` writes chosen ETP laws, together with their duals, as an equations file with the ETP's numbers as ids, so that the laws to explore can be chosen by number.

The script `etp_export.py` writes a result in the ETP's numbering and notation: its laws, its implications, simplified as in Section 2.7, each followed by its dual, and its magmas with the laws they satisfy and refute, as in the ETP's `Facts`. Two labels relate the implications to the ETP.

- With `--background-label`, each background implication is labelled with the origin of the background, such as `ETP`, `equational reasoning` (Section 3.4), or the name of an earlier exploration whose result is the background. The label gets "and reversal" ("and duality" for magmas) when its conclusion does not follow from the background implications without their duals. The background is closed under duality, and among semigroups the dual of a law is its reversal: the ETP's Eq10 $\to$ Eq3322, that is, $x = xyx$ implies $xy = xyzy$, reversed gives Eq10 $\to$ Eq3326, $xy = xzxy$, which holds in all semigroups but not in all magmas.
- With `--etp-data`, the directory that `fetch_etp.py` writes, an implication is marked `new` when the ETP's implications do not give it: some law of its conclusion follows from no

single law of its premise, nor from an axiom of the exploration, by chains of the ETP's implications and their duals (reversals, among semigroups), through any of the ETP's laws, explored or not. The ETP studies implications from a single law, so implications with several premises are usually new, as are those of explorations with axioms, which count as premises. A proved implication need not be new: nine proved implications of the semigroup laws of order at most four (Section 7) follow from ETP implications chained through laws outside that exploration, which its background did not include.

3.6 The Z3 expert

The `MagmaExpert` answers a question $P \to C$ as follows.

1. It checks a *pool* of magmas: 24 small standard magmas (projections, constants, cyclic groups, the two-element Boolean magmas, a rock–paper–scissors tournament, all two-element tables), every counterexample found so far, and any magmas given as seeds. The first magma satisfying P and violating a law of C is the counterexample.
2. Otherwise, for each size n from a minimum to a maximum (by default 1 to 6), it asks the Z3 SMT solver for a table of size n satisfying P and violating some law of C. The operation is an uninterpreted function restricted to $\{0, \ldots, n-1\}$, and the laws are grounded over all assignments. The first table found is the counterexample and joins the pool.
3. If no size yields a table, the implication is accepted. It is reported as confirmed if Z3 answered "unsatisfiable" for every size and as unconfirmed if it ran out of time (by default 30 seconds) for some size.

A "confirmed" implication thus only means that it has no counterexample of the searched sizes. It may be false, refuted only by a larger or an infinite magma, and both case studies show that this happens.

3.7 The Vampire expert

The `VampireExpert` uses the first-order theorem prover Vampire. A question $P \to C$ becomes a TPTP problem whose axioms are the universal closures of the laws of P and whose conjecture is the conjunction of the laws of C. Vampire then answers with a status: *Theorem* (it found a proof), *CounterSatisfiable* (the axioms do not imply the conjecture, shown by a finite model or by a saturated clause set), or a timeout.

The expert proceeds through the following steps and stops at the first one that decides the question.

1. **Pool and memory.** The pool is checked as by the Z3 expert. A question entailed by an implication proved before ($P' \subseteq P$, $C \subseteq C'$) is accepted, and an undecided answer is reused if the same question is asked again.
2. **Proof.** Vampire's portfolio mode (`--mode casc`) runs on the problem, on all cores, since nothing else runs at this point (option `--vampire-first-time-limit`, by default the proving time limit of 10 seconds). A proof confirms the implication. If the run instead ends with a finite model, the model is the counterexample. On the 100 whole conclusions of the benchmark described in Section 3.8, eight cores decided the same 81 as one core and none in more than 1.8 seconds, against 9.6 seconds for the slowest decision on one core; with a limit of 3 seconds, the step took 70% less time than on one core with 10 seconds, and lost no decision.
3. **Quick model search.** Vampire's finite model builder (`--saturation_algorithm fmb`) looks for a counterexample of any size (by default for 1 second).

4. **One law shown false.** If C has a single law and step 2 ended with CounterSatisfiable, the free magma over P violates that law and refutes the implication. No further call is made: the rest of its row is decided by the context already, the laws outside $P \cup C$ being denied for P by its objects.

5. **Free magma.** The row of the free magma over P is computed, as described in Section 3.8. If a law of C is on its negative side, the free magma refutes the implication. If every law of C is on its positive side, the implication is proved. If step 2 ended with CounterSatisfiable and a single law c of C is not on the positive side, the free magma violates c: a brief search (1 second) looks for a finite model violating c, and the free magma refutes the implication if it finds none.

6. **Per-law model search.** For each law c of C that the free magma did not prove, the model builder looks for a model of P violating c (by default for 10 seconds each), all searches sharing the cores. This is much faster than a search for a model violating the conjunction C, which has to consider all the ways of violating it at once. Optionally, the model finder Mace4[3] searches as well (option `--mace4-time-limit`, off by default), with its own propagation and symmetry reduction, which sometimes finds models that Vampire's model builder does not reach in time. Each Mace4 search is for one law at one size, and the sizes are swept in parallel: a core that finishes a search, having found a model or shown that there is none of that size, takes the next search, for the smallest size not yet started; the sweep gets the given time in all. A search at a fixed size where a model exists is often fast, while ruling out the smaller sizes first, as a search size by size has to, can take much longer. Each model found is checked and joins the pool, and the searches for the laws it violates stop; the others go on until every law of C is violated or the time is up. The smallest model found refutes $P \to C$, and the pool then answers the next questions about P, whose conclusions are the laws the model satisfies, without a further call. A law whose search ran out of time is not searched for again with the same premise. Among semigroups, with option `--truncation-words`, the truncated free semigroups of the variety of P are tried first (Section 9.3), which refute laws whose smallest countermodels are far beyond the model searches. With option `--model-premise simplified`, the searches use a subset of P whose closure under the background and the implications proved so far is P, keeping the laws that every object satisfies, which the background assumes rather than derives; `both` searches with both premises. Every model is checked against the whole premise.

7. **Undecided.** Otherwise, the laws of C that the free magma proved are accepted as confirmed. The rest of C, which has at least two laws if step 2 ended with CounterSatisfiable, is rejected if it did, that is, if Vampire showed $P \to C$ false as a whole, and is accepted as unconfirmed otherwise. The partial row of the free magma is not added to the context. Its positive side holds in every object of the context that satisfies P, since objects are completed with the trusted implications, and every law on its negative side lies outside C and is therefore denied by some object satisfying P; so the row could refute nothing that the context does not already refute.

Every finite model is read from the output of Vampire or Mace4 and checked against the laws; a model that does not refute the question stops the program with an error. Vampire's proofs and its CounterSatisfiable verdicts obtained by saturation are trusted. The expert counts its answers by kind (`pool`, `proved`, `model`, `free magma`, `rejected`, `unconfirmed`), and the counts are printed at the end.

Unlike the Z3 expert, the Vampire expert never accepts an implication as confirmed without a proof, and it never accepts an implication it knows to be false.

[3] `https://www.cs.unm.edu/~mccune/prover9/`

3.8 Free magmas

Let P be a set of laws. Call two terms s and t, in countably many variables, *equivalent* if P derives $s = t$. The *free magma* F_P over P is the set of equivalence classes, with the operation $[s] * [t] = [s * t]$, which is well defined because P derives $s * t = s' * t'$ whenever it derives $s = s'$ and $t = t'$. It satisfies P, and by Birkhoff's theorem it satisfies a law L exactly when P derives L. Hence:

Proposition 7. *For every finite set M of laws and every $P \subseteq M$, the row of F_P is $\{ L \in M : P \vdash L \}$, the closure of P in $\mathbb{K}_{\mathrm{Mag}}$. In particular, F_P refutes every false implication with premise P.*

For one-variable laws, the submagma of F_P generated by one variable already has the same row. F_P is usually infinite, so it has no table. Its row is computed law by law. The laws that P implies by the background implications and by the implications Vampire has proved so far are on the positive side without asking. For each other law L, Vampire runs in portfolio mode on the problem "P implies L", all laws in parallel and the laws of the conclusion first. A proof puts L on the positive side, CounterSatisfiable on the negative side, and a timeout leaves L unknown. With `--eprover-time-limit`, the E prover[4] decides each law of the conclusion first, with its automatic strategy and the given time limit, and Vampire is asked only about the laws that E leaves undecided. With `--prover9-time-limit`, Prover9[5] then tries to prove each law of the conclusion that E left, before Vampire; only its proofs count, since its search failing does not show a law false. Different provers find different proofs easy: Prover9 proved in 2 seconds the one implication of Section 9 that Vampire left open, on which Vampire needed 16 seconds on four cores. On 500 such problems drawn from the questions of the fourth exploration of Section 6, E decided 453 and Vampire 445, E decided 10 that Vampire did not and Vampire 2 that E did not, and the two never contradicted each other. When E decided, it did so within 0.34 seconds in 99% of the cases and never took more than 1.04 seconds, whereas it used its whole limit when it did not; with 2 seconds for E first, the step cost half as much processor time as with both provers running in full, and no decision of the sample was lost. On the whole conclusions of the first step, E decided nothing that Vampire did not, so it is not used there. E's saturation sometimes succeeds where Vampire's does not: in the fourth exploration of Section 6, it showed within a second that none of the 60 laws of the conclusion of an unconfirmed implication follows from its premise, which Vampire failed to show in 30 seconds per law even with its schedule for satisfiable problems. The row is then completed with the known implications, as the context completes its objects: the laws that the positive side implies are positive, and a law that would imply a law on the negative side is negative. The laws of the conclusion get the proving time limit (by default 10 seconds); the others, which decide nothing about the question but complete the row, get a separate *row time limit* (option `--vampire-row-time-limit`, by default the proving time limit). With row time limit 0, only the laws of the conclusion are asked about, and the other unknown laws stay unknown unless completion fills them in.

The row time limit matters when many questions end with a free magma that Vampire cannot fully decide: each of them then costs the row time limit for the laws outside the conclusion, which run in parallel on all cores. With the 149 laws of Section 6, a premise that implies few of them leaves well over a hundred laws to ask about, so that every such question costs about the row time limit times the number of laws divided by the number of cores: about 3 minutes on 8 cores with the default 10 seconds, and under 20 seconds with a limit of 1 second.

A free magma is a powerful object: when its row is complete, its intent is the closure of P in $\mathbb{K}_{\mathrm{Mag}}$, so it refutes every later question whose premise it satisfies and whose conclusion is not derivable. In the results, free magmas are named `F`k, the dual of F_P being the free magma over

[4] `https://eprover.org`

[5] `https://www.cs.unm.edu/~mccune/prover9/`

the dual laws of P. The negative side of a free magma rests on Vampire's CounterSatisfiable verdicts. Those obtained from a finite model could be checked; those obtained by saturation cannot be checked by the library and are trusted.

3.9 The completion expert

With `--expert completion`, a question $P \to C$ is decided by Knuth–Bendix completion of the laws of P into a convergent rewrite system R (Section 5 describes how such a system certifies non-implications). If the two sides of every law of C have the same normal form, the implication is accepted as proved. Otherwise the free magma over P refutes it, and its row is complete: the normal forms decide every law. The system of each premise is checked as a certificate once, so that every negative entry is certified; positive entries rest on the soundness of completion. When completion fails, on an equation that the order cannot orient, such as commutativity, or at its limits of 400 rules and 10 seconds, the question goes to the Vampire expert if Vampire is installed, and is otherwise accepted as unconfirmed. The expert is meant for one-variable laws, for which completion has always succeeded; for laws in several variables it often fails. It has no pool of finite magmas, so `--seed-magmas` does not apply to it.

3.10 Seeding with known magmas

Magmas found in earlier explorations (the `magmas.json` of saved results) can be reused in two ways, and so can the finite magmas of the ETP: files with one table such as `[[0, 1], [1, 0]]` per line, after the word `Table` or an equation number, as in the ETP's `equational_theories/Generated/All4x4Tables/data` and `data/smallest_magma_examples.txt`, are read as well (repeated tables once). They can be added to the expert's pool, which is checked before any solver runs. Alternatively, they can be added to the context before the exploration starts: they are evaluated on every law, and objects whose rows the others account for are dropped (an object is redundant if, for each law it violates, another remaining object satisfies everything it satisfies and violates that law). No question that the seeded objects refute is then asked.

4 Case study: sixteen classical laws

The file `equations.json` lists the sixteen laws of Table 1. Some of them have two or more variables, so no background implications are derived; the only prior knowledge is the duality symmetry.

4.1 Two explorations

The laws were explored twice, once with each expert, with the default settings (Table 2).

The Vampire exploration proved every implication it accepted and refuted every other question with a checked finite magma, so its theory is exactly the set of implications between these laws that hold in all magmas. Its implications with a single premise agree with the ETP (Section 3.5): fifteen of the sixteen laws, all but mediality, are ETP laws, and for each of the 210 ordered pairs of distinct ones, the theory contains the implication from the first to the second exactly when the ETP graph does. Its simplified implications are listed in Table 3; each stands also for its dual, which swaps Eq4 and Eq5, Eq10 and Eq25, Eq16 and Eq26, Eq323 and Eq378, Eq4396 and Eq4473, and fixes the other laws.

4.2 Where the experts differ

The Z3 exploration accepted three implications that are false: one implication together with its dual, and one self-dual implication.

Law	Name	Equation
Eq2	Singleton	$x = y$
Eq3	Idempotence	$x = x * x$
Eq4	Left-zero	$x = x * y$
Eq5	Right-zero	$x = y * x$
Eq10	Left-central identity	$x = x * (y * x)$
Eq16	Steiner law 1	$x * (x * y) = y$
Eq25	Central identity	$x = (x * y) * x$
Eq26	Steiner law 2	$(y * x) * x = y$
Eq43	Commutativity	$x * y = y * x$
Eq323	Left-idempotent composition	$x * y = x * (x * y)$
Eq378	Right-idempotent composition	$(y * x) * x = y * x$
Eq4396	Left-alternative	$(x * x) * y = x * (x * y)$
Eq4435	Flexible	$(x * y) * x = x * (y * x)$
Eq4473	Right-alternative	$(y * x) * x = y * (x * x)$
Eq4512	Associativity	$(x * y) * z = x * (y * z)$
Medial	Medial (entropic)	$(x * y) * (z * w) = (x * z) * (y * w)$

Table 1: The laws of `equations.json`. The names are those of the file.

	Z3, sizes 1–6	Vampire, 10 s / 1 s / 10 s
Questions	59	62
Implications accepted	27	27
of which unconfirmed	2	0
Implications found (with images)	43	44
Counterexamples	32	35
Free magmas	—	0
Rejected implications	—	0
Running time	2 min 52 s	2 min 51 s

Table 2: The two explorations of the sixteen laws. Of Vampire's 35 counterexamples, 27 are models found by Vampire and 8 come from the pool. The Vampire time limits are those for a proof, the quick model search and the per-law model search.

- Eq16, Eq4435, Eq4473 → Eq4396, Eq4512, Medial (and its dual). Only Eq4396 follows (implication 11). Associativity and mediality fail in a Steiner loop of order 10: a commutative loop in which every element is its own inverse, built from the Steiner triple system on nine points (the affine plane of order 3)[6]. It satisfies Eq16, Eq26, Eq43, Eq4396, Eq4435 and Eq4473 but none of the other laws.
- Eq3, Eq16, Eq26, Eq43, Eq4435 → Medial. It fails in the Steiner quasigroup of order 7, the idempotent commutative quasigroup of the Fano plane, in which $x * y$ is the third point on the line through x and y[7].

Both counterexamples exceed the sizes the Z3 expert searches. Conversely, the two implications Z3 accepted as unconfirmed (Eq323, Eq378, Medial → Eq4396, Eq4435, Eq4473, Eq4512, implication 5 before simplification, and Eq43, Eq4396, Eq4435, Eq4473, Eq4512 → Medial) are true, and Vampire proved them. Every implication Vampire accepted is entailed by the Z3 theory, as it should be.

[6] To check: the library only checked that the magma is a commutative loop with $x * x = e$ for its identity e and which laws it satisfies; the identification relies on the correspondence between Steiner loops of order $n + 1$ and Steiner triple systems on n points, and on the uniqueness of the system on nine points.

[7] To check: the library only checked which laws the magma satisfies; the identification relies on idempotent totally symmetric quasigroups being Steiner quasigroups and on the uniqueness of the Steiner triple system on seven points.

1	Eq16, Eq26 → Eq43	15	Eq10 → Eq378
2	Eq4512 → Eq4435, Eq4473	16	Eq10, Eq323 → Eq4473
3	Eq26, Eq378 → Eq4	17	Eq10, Eq4435 → Eq25
4	Eq16, Eq378 → Eq10, Medial	18	Eq10, Eq4473 → Eq3
5	Eq323, Eq378, Medial → Eq4512	19	Eq10, Eq4396 → Eq323
6	Eq43 → Eq4435	20	Eq5 → Eq4512, Eq10, Eq16
7	Eq26, Eq43 → Eq16	21	Eq3, Medial → Eq4435
8	Eq43, Eq378 → Eq323	22	Eq3, Eq378 → Eq4473
9	Eq26, Eq4473, Medial → Eq4512	23	Eq3, Eq4473 → Eq378
10	Eq16, Eq4473, Medial → Eq4512	24	Eq10, Eq25 → Eq4435
11	Eq16, Eq4435, Eq4473 → Eq4396	25	Eq25, Eq4512 → Medial
12	Eq43, Eq4473 → Eq4396	26	Eq25, Eq43 → Eq2
13	Eq26, Eq4512 → Medial	27	Eq2 → everything
14	Eq43, Eq4512 → Medial		

Table 3: The implications between the sixteen laws, as the simplified implications printed by the program. Together with their duals, they entail every implication that holds in all magmas.

The first of the false implications also illustrates the per-law model search of step 6. Before that step was added, the Vampire expert left this question undecided: the free magma over the premise decided neither associativity nor mediality in time, and a model violating their conjunction was not found within the time limit. The model builder finds the size-10 Steiner loop in a few seconds when it looks for a violation of a single law, while the search for a violation of both needs more than a minute and a half.

5 Case study: one-variable laws of order at most four

The file `equations_1var_order4.json` lists the 31 laws in one variable of order at most four, the trivial law Eq1: $x = x$ included. Up to the order of the sides, they are all the equations $s = t$ between the 23 one-variable terms with at most four operations such that the two sides together have at most four operations: 22 laws $x = t$, seven laws $x * x = t$, the law $x * (x * x) = (x * x) * x$, and Eq1. Since all laws have one variable, background implications can be derived.

5.1 Background

With order $n = 4$, transient order $n' = 10$ and premise size $K = 2$, the derivation closure yields 111 background implications. With premise size 0, the canonical basis of the derivation closure has 95 implications; the two sets entail each other, so for these laws premises of size two lose nothing. The 111 implications have 108,635 closed sets.

5.2 Results

Table 4 compares two Vampire explorations, which differ only in the order of the steps of the expert, with two explorations by the Z3 expert.

No implication beyond the background. Both Vampire explorations refuted every question: no implication was accepted, and the background alone generates the theory. This is a mathematical result: *every implication between one-variable laws of order at most four that holds in all magmas follows from the 111 background implications*, that is, it can be derived by equational reasoning through equations of order at most four, with single steps reaching order ten, from premises of at most two laws. Conversely, every non-implication is witnessed either by a finite magma, checked by the library, or by a free magma whose relevant negative

	Vampire		Z3, sizes 1–6	
	full search first	quick search first	no background	background
Background implications	111	111	—	103
Questions	17,835	18,655	8,189	8,342
Implications accepted	0	0	887	842
of which false	0	0	837	837
Refuted by a finite model	15,894	14,688	7,302	7,500
Refuted by a free magma	1,941	3,967	—	—
Rejected or unconfirmed	0	0	0	2
Closed sets of the theory	108,635	108,635	75,127	75,127
Running time	8 h 05 min	2 h 25 min	not recorded	3 h 18 min

Table 4: Explorations of the 31 one-variable laws. "Full search first" runs a 10-second finite model search on the conjunction before computing the free magma; "quick search first" runs a 1-second search, then the free magma, then a 10-second search per law. "Refuted by a finite model" includes the 7 questions answered by the pool. The Z3 exploration with background used order 7 and no transient order. The Vampire running times are wall-clock times on an 8-core laptop; the second excludes a 40-minute sleep of the machine.

entry Vampire found by saturation and Knuth–Bendix completion certifies (see below). The implications with a single premise agree with the ETP (Section 3.5): all 31 laws are ETP laws, and for each of the 930 ordered pairs of distinct laws, the background implies the second from the first exactly when the ETP graph does. The final context of the second run has 37,177 objects (originals and duals) with pairwise distinct rows, and its 108,635 intents are exactly the closed sets of the background. Every row was decided completely: no free magma has an unknown entry.

Certifying the free magmas. The negative entries of the free magmas need not rest on Vampire. For laws P and $s = t$, a rewrite system R shows that P does not imply $s = t$ if every rule of R decreases in the Knuth–Bendix order, every critical pair of R is joinable, every law of P is joinable in R, and s and t have different normal forms: R is then convergent, entails P, and does not entail $s = t$. These conditions are checked on R itself, however it was found. The script `completion.py` finds R by Huet's completion of P with the Knuth–Bendix order of weight 1, and checks it. For the first Vampire exploration, completion terminated for every one of the 3,858 premises of its free magmas (originals and duals), within six seconds in all, and certified all 80,886 negative entries; the certificates are saved with its results (`free_magma_certificates.json`). It also certified all 167,042 negative entries of the 7,910 free magmas of the second exploration. As a check of the checker, none of the 38,712 positive entries of the free magmas of the first exploration is separated, and deciding by completion each of the 930 implications with a single premise between the 31 laws agrees with the ETP graph. With completion, the non-implications of this case study thus rest on checked finite magmas and checked rewrite systems, and no longer on Vampire.

Exploration by completion. The completion expert (Section 3.9) decides every question of this case study by itself: two explorations with it, one with the background above and one without any background, never needed Vampire. With the background, it asked 54,575 questions, refuted all of them with free magmas and accepted nothing, in 8 min 15 s; without background, it asked 54,618 questions, refuted 54,568 with free magmas and proved 50 implications, 95 with their duals, in 8 min 57 s. Both theories entail every implication of the Vampire explorations and conversely, and have the same 108,635 closed sets, so this case study's result is confirmed by an expert that uses neither Vampire's proofs nor its saturation nor finite models. The

explorations ask about three times as many questions as the Vampire ones, since the row of the free magma over P is exactly the closure of P, so that it refutes only the question it answers and its dual, whereas a finite magma often refutes many; but each answer takes milliseconds.

The counterexamples. The finite counterexamples found in the first Vampire run have the following sizes (originals only):

Size	2	3	4	5	6	7	8	9	10
Magmas	5	92	876	2,741	2,837	2,247	4,694	2,200	202

Most counterexamples are larger than six elements, and some implications have no finite counterexample that Vampire finds in time; for those the free magma is the counterexample. A typical example is question 685 of the second run, whose premise consists of Eq1 and seven laws including $x = (((x * x) * x) * x) * x$ and $x * x = x * (x * (x * x))$: the free magma over the premise satisfies exactly the eight premise laws and refutes all 23 others at once.

What the Z3 explorations got wrong. The Z3 expert searches only up to six elements, and its explorations accepted 887 and 842 implications. Only 50 and 5 of them, respectively, follow from the background (the second exploration already had most valid implications as background); the remaining 837 are false in both cases. The false implications made the enumeration skip about a third of the closed sets, which is why the Z3 explorations asked fewer than half as many questions. In this domain, bounded finite-model search is a poor expert: an implication without small counterexamples is more often false than true.

Order of the steps. In the first Vampire exploration, a 10-second model search on the conjunction preceded the free magma. Most questions that ended with a free magma thus first spent 10 seconds in a hopeless search. Running a 1-second search first, the free magma next, and a 10-second search per law last cut the running time by a factor of more than three. The number of questions differs between the runs because different counterexamples were found, which changes the context and therefore which closed sets need a question; the theory is the same.

6 Case study: ETP laws of order at most two and three

The ETP has decided every implication $E \to F$ between two of its laws, but an implication with a larger premise, such as $E_1, E_2 \to F$, is not determined by those: E_1 and E_2 may together imply a law that neither implies alone. Attribute exploration with the ETP graph as background (Section 3.5) asks exactly about such premises. The two explorations of this section apply it to the ETP laws of order at most two and three.

Implications whose premise has several laws are among the future directions listed in the ETP paper [ETP25, Section 14, item (2)], and were discussed on the ETP's Zulip channel from its start. There, Bruno Le Floch found with Prover9 that the conjunctions of the laws of order at most two fall into 59 equivalence classes, and computed all implications between them[8]. He has since determined all equivalence classes of conjunctions, and the implications between them, for 990 quasigroup laws [LF26a] and for semigroup laws of order up to four [LF26b], and reports in the latter that the implication semilattice of conjunctions of magma laws of order up to three, with a few thousand equivalence classes, is almost determined in work in progress by other members of the ETP. The explorations of this section are therefore independent computations of known or nearly known results, by a different method: attribute exploration, which yields

[8] `https://leanprover-community.github.io/archive/stream/458659-Equational/topic/Search.20space.20for.20(EqX.20.E2.88.A7.20EqY).20.E2.86.92.20EqZ.html`

the canonical basis of the theory relative to the ETP background, and with the implications of order at most two proved in Lean.

6.1 Choice of the laws

Laws that imply each other are interchangeable in an implication, so it suffices to explore one law from each class of equivalent laws. The ETP lists 46 laws of order at most two, in 23 classes, and 410 laws of order at most three, in 146 classes. One law is chosen from each class: the law with the smallest number, or, if the class contains a law equal to its own dual, the first such law. If no law of a class is self-dual, the dual of the chosen law, which lies in the dual class (possibly the same class), is chosen as well and represents that class, so that the set is closed under duality. This gives 24 laws of order at most two (`equations_order2.json`) and 149 of order at most three (`equations_order3.json`). Both files were written by `etp.py`, which computes the classes from the ETP graph and lists the members of each representative's class in the file:

```
python explorations/equational_theories/etp.py equations.txt \
    --max-order 3 --graph graph.json -o equations_order3.json
```

Each set includes Eq1: $x = x$ and Eq2: $x = y$, which hold in every magma and only in the trivial magma, respectively. The two explorations took 18 and 138 background implications from the ETP graph.

6.2 Order at most two

With the Vampire expert and the ETP background, the exploration of the 24 laws asked 45 questions. It found 19 counterexamples, 6 of them two-element magmas of the pool and 13 found by Vampire, of sizes 3 to 5, and proved 26 implications; nothing was left unconfirmed or rejected, and no free magma was needed. The theory has 59 closed sets, and each of its 26 simplified implications has a premise of two laws. For example:

- $\mathrm{Eq25}, \mathrm{Eq43} \to \mathrm{Eq2}$: a commutative magma satisfying $x = (x * y) * x$ is trivial;
- $\mathrm{Eq26}, \mathrm{Eq29} \to \mathrm{Eq43}$: the laws $x = (x * y) * y$ and $x = (y * x) * y$ together imply commutativity;
- $\mathrm{Eq25}, \mathrm{Eq8} \to \mathrm{Eq3}$: the laws $x = (x * y) * x$ and $x = x * (x * x)$ together imply idempotence.

Since every question was decided, this is the complete implicational theory of the ETP laws of order at most two: every implication between them that holds in all magmas follows from the 26 simplified implications, the 18 single-premise implications of the ETP graph, duality, and the equivalence of the laws of each class. Its 59 closed sets agree with Le Floch's 59 equivalence classes of conjunctions.

6.3 Order at most three

The 149 laws have far more closed sets, and many premises combine laws whose consequences Vampire finds hard to decide. Four explorations were run, all with the ETP background, the context seeded with the ETP's finite magmas (Section 3.5; 233 of 1374 tables were kept), a row time limit of 1 second, and otherwise the default limits. The second also used Mace4 in the per-law model search, for 30 seconds per law, the third, in addition, the ETP's refutations as objects, and the fourth, in addition, asked about the duals of unconfirmed implications instead of adding them (Section 2.4). Table 5 compares them. With the ETP data fetched into `etp` (Section 3.5), the fourth was run as

```
python explorations/equational_theories/explore.py \
    --equations explorations/equational_theories/equations_order3.json \
    --expert vampire --vampire-row-time-limit 1 --mace4-time-limit 30 \
    --etp-graph etp/graph.json --etp-equations etp/equations.txt \
    --etp-refutations --ask-unconfirmed-duals --seed-context etp/magmas/*.txt
```

	Vampire	and Mace4	and refutations	and duals asked
Questions	1,310	1,326	1,337	1,376
proved	736	746	746	748
refuted by a finite model	437	440	444	444
refuted by a free magma	75	76	72	74
unconfirmed	57	59	70	105
rejected	5	5	5	5
Accepted implications	807	802	824	860
confirmed	750	754	765	767
unconfirmed	57	48	59	93
of which known to be false	6 or more	3	1	1
Canonical basis relative to the background, up to duality	793	795	806	806
of the confirmed implications alone	750	754	765	767
Closed sets of the theory	3,650	3,663	3,667	3,671
without the unconfirmed implications	29,415	28,927	11,107	11,097
Rounds	1	2	2	2
Running time	2 h 48 min	4 h 34 min	4 h 42 min	6 h 19 min

Table 5: Explorations of the 149 representatives of the ETP laws of order at most three, with the ETP graph as background and the ETP's finite magmas in the context: by the Vampire expert, with Mace4 in the per-law search as well, with the ETP's refutations in the context as well, and asking about the duals of unconfirmed implications as well. The questions are counted by answer, so that a question asked again in a later round counts twice. The accepted implications are those the expert accepted, without the background and without the duals that the symmetry adds; in the fourth exploration, the duals of unconfirmed implications are asked about and accepted in their own right, so that its 93 unconfirmed implications are 60 up to duality. Those known to be false were shown false by later explorations or tests, and they remain in their exploration's result. The canonical basis is that of the theory relative to the background, counted up to duality: in all, it has 1,552, 1,557, 1,579 and 1,579 implications, of which 33 or 34 have self-dual premises. The second line gives the canonical basis, relative to the background and up to duality, of the theory that the confirmed implications and their duals generate. In the first three explorations, the simplified implications printed by the program (Section 2.7) are as many as the canonical basis up to duality; in the fourth, which does not map unconfirmed implications by duality, they are 842, its 93 unconfirmed implications being 60 up to duality. The closed sets without the unconfirmed implications are counted without their duals as well, but with the duals of the confirmed implications, a few of which the fourth exploration did not record because unconfirmed implications entailed them. The running times are wall-clock times on an 8-core laptop; the second excludes a sleep of the machine of about an hour.

Where Vampire gets stuck. Most hard questions involve Eq255: $x = ((x * x) * x) * x$ or its dual Eq47: $x = x * (x * (x * x))$, combined with laws in two or more variables. In the second exploration, 44 of the 48 unconfirmed implications and all five rejected implications have one of

these two laws in their premise. Their premises are often large: the rejected implications have 16 to 18 laws in their premise.

Testing the unconfirmed implications. After the first exploration, each law of the conclusion of each of its 19 unconfirmed questions (109 laws in all) was tried again, separately, with longer limits:

- Vampire, 60 seconds in portfolio mode and 60 seconds with its model builder, first on the premise alone and then with the one-variable laws that the premise derives (Section 3.4) added as lemmas: none of the 109 laws was decided;
- Prover9, 60 seconds: no proof;
- Mace4, 60 seconds: models of size 16 violating 34 of the laws, found within 0.2 to 31 seconds. These settled six of the 19 questions as false, all their unproved laws being violated, and three magmas sufficed for all six.

So at least six of the 57 unconfirmed implications of the first exploration are false, and the enumeration, which they steered, skipped closed sets. Its 3,650 closed sets are therefore not those of the true theory.

The second exploration. With Mace4 in the per-law search, the expert found the counterexamples of size 16 during the exploration itself, together with models of sizes 11 and 13 that Vampire had missed (17 tables in all, counting duals). One of them refuted an unconfirmed implication accepted 260 questions earlier; the exploration resumed at that question (Section 2.6), which made it run in two rounds. In the end, 48 unconfirmed implications and 5 rejected implications remain, and 46 of the 48 have the same premise as an unconfirmed implication of the first exploration. No counterexample to them is known, but they are not proved either.

Infinite counterexamples. Three of these 48 unconfirmed implications are nevertheless false:

- $\mathrm{Eq1}, \mathrm{Eq255}, \mathrm{Eq261} \to \mathrm{Eq307}$;
- $\mathrm{Eq1}, \mathrm{Eq255}, \mathrm{Eq274} \to \mathrm{Eq47}, \mathrm{Eq99}, \mathrm{Eq118}, \mathrm{Eq203}, \mathrm{Eq222}$;
- $\mathrm{Eq1}, \mathrm{Eq47}, \mathrm{Eq99}, \mathrm{Eq203}, \mathrm{Eq229}, \mathrm{Eq255} \to \mathrm{Eq73}, \mathrm{Eq125}$.

In each, one law of the premise (Eq261, Eq274 and Eq229) implies the others, and the ETP refutes that it implies some law of the conclusion: $\mathrm{Eq261} \not\Rightarrow \mathrm{Eq307}$, $\mathrm{Eq274} \not\Rightarrow \mathrm{Eq47}$, and $\mathrm{Eq229} \not\Rightarrow \mathrm{Eq73}$. The ETP's counterexamples are infinite: for the duals Eq65 (the Asterix law of the ETP), Eq73 and Eq118 of these laws, it builds them greedily on $\mathbb{N}$ or on a free group. For the first and the third implication, no finite counterexample exists, since the ETP proves in Lean that the duals of their single-premise refutations hold in finite magmas (`Finite.Equation65_implies_Equation359`, `Finite.Equation118_implies_Equation274` and `Finite.Equation118_implies_Equation222`); Mace4 also finds no magma of size at most 40 that satisfies Eq261 and violates Eq307. Whether the second holds in finite magmas was not checked. No finite search could have refuted the first and the third, and since they steered the enumeration, the theory of the second exploration is not the theory of all magmas even if its other unconfirmed implications are true.

The third exploration. The third exploration added the ETP's refutations to the context (Section 3.5). Of the 20,916 implications between two of the 149 laws that the ETP refutes, 10 remained after the reduction against the 233 magmas, five and their duals: $\mathrm{Eq261} \not\Rightarrow \mathrm{Eq307}$, $\mathrm{Eq274} \not\Rightarrow \mathrm{Eq47}, \mathrm{Eq99}, \mathrm{Eq203}$ and $\mathrm{Eq229} \not\Rightarrow \mathrm{Eq73}$, exactly those behind the three

false implications. They were not accepted. Instead, the exploration asked about the larger premises that they had covered, and accepted seven implications, counting duals, that the second theory does not entail, all unconfirmed, such as $\mathrm{Eq1}, \mathrm{Eq255}, \mathrm{Eq274}, \mathrm{Eq280} \to \mathrm{Eq43}, \mathrm{Eq73}, \mathrm{Eq75}, \mathrm{Eq125}, \mathrm{Eq229}, \mathrm{Eq264}$. Apart from these and the three false implications with their duals, each theory entails every implication of the other. The number of unconfirmed implications grew from 48 to 59, since each false implication stood for several finer questions that the expert could not decide either.

The fourth exploration. In the first three explorations, the dual of each unconfirmed implication was added with it and steered the enumeration as well. The fourth asked about these duals instead (Section 2.4). It asked every question of the third exploration and 38 more, of which 35 were left unconfirmed, mostly the duals themselves, so that it took 1 h 37 min longer. Its answers to the common questions were the same but for one: question 262, $\mathrm{Eq1}, \mathrm{Eq255}, \mathrm{Eq257}, \mathrm{Eq263}, \mathrm{Eq307}, \mathrm{Eq310} \to \mathrm{Eq260}, \mathrm{Eq266}, \mathrm{Eq323}$, unconfirmed in the third exploration, was refuted by a magma of size 13 that Vampire found within its 10 seconds this time. The third theory therefore contains a false implication and its dual; the fourth proves the rest of that question, Eq323 added to the premise, and confirms one large implication more. Apart from these, the two theories are the same, and the fourth found no unproved implication that the third theory lacks. Asking about the duals thus made the result slightly more complete in this case, but mainly by the difference at question 262 rather than by easier questions.

Posing the questions differently. The unconfirmed questions of the fourth exploration were also put to the provers in other forms, with the expert's time limits, to see whether the form of a question matters:

- with minimal premises, the fewest laws whose closure under the background and confirmed implications is the premise (2.4 laws instead of 13.6 on average): no proof among the 1,652 unproved laws, and no counterexample among 222 of them sampled; on 444 refuted questions, the full premises found three large models (sizes 15 and 16) that the minimal ones missed;
- with premises relaxed to the laws not added by unconfirmed implications: only one question had such laws, and it was not refuted;
- with the other members of the ETP equivalence class of each unproved law as the goal (1,865 goals for 294 laws), and the classes of the premise laws added to the premise: no proof and no counterexample.

Longer runs on single questions did not help either: the conjecture that, given Eq3 and Eq263, Eq385 is equivalent to $\mathrm{Eq117} \wedge \mathrm{Eq159}$, of which one half is proved, resisted Vampire for 10 minutes, Twee for 30 minutes and Mace4 up to sizes 15 and 16.

What the results say. In the fourth exploration, the 767 confirmed implications, together with the background and their duals, are proved. Every non-implication witnessed by a finite magma is checked, those witnessed by free magmas rest on Vampire's saturation, and those witnessed by the ETP's refutations rest on the ETP. The 3,671 closed sets of the theory would be correct if the 93 unconfirmed implications were true, but two of them, an implication and its dual, are false: the E prover showed that none of the 60 laws of their conclusions follows from their premises (Section 3.8). E decided none of the laws of the other 91 within 10 seconds each, and showed for each of the 5 rejected implications that none of their conclusion laws follows, so that free magmas refute them. The confirmed implications, the background and their duals generate a theory with 11,097 closed sets, an upper bound on the number of closed sets of the true theory, since each confirmed implication holds. None of the 93 is refuted by a

single-premise refutation of the ETP, which are all in the context, and whether they hold in finite magmas is not known. Settling the unconfirmed implications and finding counterexamples to the rejected ones needs either proofs that Vampire and Prover9 do not find within a minute, or counterexamples that Vampire and Mace4 do not find: finite magmas beyond their reach, or free magmas whose rows Vampire cannot decide. A comparison with the semilattice of the work in progress mentioned in [LF26b] would settle many of them, or reveal errors on either side.

7 Case study: semigroup laws of order at most four

Le Floch [LF26b] determined all implications between conjunctions of the laws of order at most four on semigroups: the 653 ETP laws whose two sides are right-associated, which are the ETP's forms of the laws on an associative operation, fall into 122 classes of equivalent laws and generate 456 theories ("associative theories"), found with Prover9, Mace4 and Vampire, with the proofs formalized in Lean. This section reproduces that result by attribute exploration, from the ETP's data alone.

7.1 Setup

Objects. Option `--axiom 4512` makes every object a semigroup: associativity is concluded from the empty premise in the background, so that every question's premise contains it, and of the ETP's 1374 finite magmas only the 24 associative ones are kept, of which the 12 that are not redundant are seeded into the context. The duality compares laws as words (Section 3.3).

Laws. The 653 laws were selected from the ETP's list by the syntactic condition alone, both sides right-associated and order at most four; their number agrees with Le Floch's. The ETP graph, whose implications hold in all magmas and so in all semigroups, splits them into 219 classes. Its equivalences are only those that hold without associativity; the ones that associativity adds, which lead to Le Floch's 122 classes, are left to the exploration. One law of each class is enough, but the reversal of a law's words, its dual among semigroups, is equivalent to its mirror image only modulo associativity and need not lie in the class of that image, so the 219 first laws of the classes are not closed under duality. Adding their reversals (28 laws) and associativity gives the 248 laws of `equations_semigroup.json`, written by `etp.py --max-order 4 --semigroup` (Section 10). The ETP graph gives 228 background implications between them.

Expert. The Vampire expert ran with the settings of `configs/semigroup.toml`: 3 seconds on all cores for the first proof attempt, E for 2 seconds before Vampire in the free magma's row, row time limit 0, and, in the per-law model search (Section 3.7), 300 seconds for Vampire's model builder per law and 600 seconds for Mace4's size sweep, both with simplified premises:

```
python explorations/equational_theories/explore.py \
    --config explorations/equational_theories/configs/semigroup.toml
```

7.2 Results

Every question was decided (Table 6): the 204 accepted implications are proved, and the 47 counterexamples are finite semigroups, checked against every law, so that no answer rests on a prover's saturation.

Laws explored (of the 653, with associativity)	248
Background implications from the ETP graph	228
Questions	251
proved	204
refuted by a finite model found for the question	40
refuted by a model in the pool	7
refuted by a free magma, unconfirmed or rejected	0
Canonical basis relative to the background and duality	204
relative to the ETP graph's implications alone, up to duality	214
relative to no background, up to duality	308
Closed sets of the theory	456
Rounds	1
Running time	18 min

Table 6: The exploration of the semigroup laws of order at most four. The 47 counterexamples are semigroups of sizes 3 to 12, 16, 27 and 32, found by Vampire's model builder (30) and Mace4 (17); seven of them, found while the search went on for other laws of a question, answered a later question from the pool. Duality acts on the background as well: the dual of an implication of the ETP graph between right-associated laws reverses their words, holds only in semigroups and is not in the graph, and the exploration adds these duals, 86 implications, to the background it works with. Relative to that background, and with every question decided in one round, the canonical basis up to duality consists of the 204 accepted implications (336 implications in all, 72 of them self-dual). Relative to the graph's implications and associativity alone, it has 214 up to duality (343 in all), and relative to no background, for the whole theory, 308 (508 in all). The program prints 204 simplified implications. The running time is the wall-clock time on an 8-core laptop.

Comparison with Le Floch's result. A law of the 653 holds in a closed set of the exploration when the first law of its ETP class does. So counted, the 456 closed sets are exactly Le Floch's 456 theories, and the classes of laws with the same closure are his 122, except that the exploration puts $x = x$ and associativity in one class, associativity being assumed. The exploration thus reproduces the classes, the theories and the implications between them without using any of his data.

Every theory has a model, and the implications are the whole propositional theory. Of the 456 closed sets, 51 are meet-irreducible (38 up to duality, 25 of them self-dual), and each of them is the intent of a finite semigroup of the context, of 2 to 32 elements; the 75 distinct intents of the context's 79 objects include these 51, and the other 24 are intersections of them. A direct product of semigroups satisfies exactly the laws that all its factors satisfy. So every closed set, the intersection of the meet-irreducible ones containing it, is the intent of the product of their semigroups, and the full set that of the trivial semigroup: in the context of all semigroups, every closed set is an object intent, that of a finite semigroup that satisfies exactly the laws of the theory. The object intents of that context are therefore closed under intersection and contain the full set, so that they are exactly the models of their Horn theory. Every propositional formula over these laws that holds in all semigroups thus follows from the implications: no clause $\neg p_1 \vee \cdots \vee \neg p_k$ holds, the trivial semigroup satisfying every law, and $p_1 \wedge \cdots \wedge p_k \to q_1 \vee \cdots \vee q_m$ holds only if $p_1 \wedge \cdots \wedge p_k \to q_j$ holds for some j, since semigroups violating each q_j while satisfying the p_i have a product violating all of them. The implications found are thus not only the Horn envelope of the theory of semigroups over these laws but that theory itself. Magmas are closed under products as well, possibly infinite ones among their factors, so that for the laws of the previous sections, too, the theory of all magmas is determined by its implications, and only the implications need to be found; where an exploration leaves

implications unconfirmed, its implications are of course not known to be that theory.

The hardest questions. Four questions needed large models. Two have the premises of theories that Le Floch singles out [LF26b, Section 4]: the premise of question 216 generates the theory of Eq56 ∧ Eq75, whose smallest non-commutative model has 27 elements, and that of question 159 the theory of Eq440 ∧ Eq513, whose smallest model violating Eq504, $x = y(y(x(yy)))$, has 32. Mace4's sweep found both models. Questions 208 and 233 needed models of 16 elements, found by Mace4 and by Vampire's model builder.

7.3 How the model search was extended

The per-law model search of Section 3.7 took its present form during this case study. A first exploration of Le Floch's 122 class representatives with associativity, without duality and with the former per-law search (10 seconds for Vampire's model builder and 30 for Mace4, both trying the sizes in increasing order, on the full premise), accepted three false implications as unconfirmed, which, written as words,

- $xx = xyzy$, $xyz = xzy \to xx = xyzw$;
- $xx = yyxx$, $xyy = yxx$, $xyz = zyx \to xx = yxzw$;
- $xx = yzyx$, $xyz = yzx \to xx = yxzw$,

for which the premises are given in simplified form. They hid 8 of the 456 theories, and two true implications were never asked. One semigroup of 16 elements refutes all three; for the third premise, Mace4 shows that no smaller one exists. Separate tests showed what finds such models:

- *A fixed size.* Mace4 searching sizes in increasing order reached size 13 in 15 minutes without a model, while at size 16 alone it found one in 213 seconds. Ruling out the sizes below the smallest model is what takes the time, roughly doubling from one size to the next.
- *The simplified premise.* At size 16, Mace4 needed 213 seconds with the simplified premise of the first implication and 897 with its full premise of 25 laws; for the third implication, Vampire's model builder found a model in 112 seconds with the simplified premise and none in 15 minutes with the full one, which it left at size 8. For magmas of order three, the full premise had done slightly better (Section 6), hence the option.
- *Not saturation.* E and Vampire, 10 minutes each, saturated none of the three problems, so longer proof attempts in the free magma's row would not have helped.

Larger sizes are not hopeless for laws whose two sides are products, such as those of the three implications: adding a copy of an element, with the same row and column, preserves such a law and its failures, so that a counterexample of size n gives counterexamples of every larger size, and a search at any size at least the smallest succeeds. The size sweep exploits this on all cores without guessing the size, and the search goes on for the laws a model does not violate, so that the pool answers the next questions about the same premise. With these changes and duality, an exploration of Le Floch's representatives together with the reversals of 11 of them, which his classes contain, already gave his 456 theories; the exploration of this section chooses its laws from the ETP's classes instead.

8 Case study: commutative semigroup laws of order at most five

The ETP's laws stop at order four. Among commutative semigroups, the laws of order five are few enough to explore: this section explores them with the result of Section 7 as background, and shows how questions whose counterexamples are too large to search for were settled by constructing the counterexamples instead.

8.1 Setup

Laws. Among commutative semigroups, a term is determined by how often each variable occurs in it, so a law is a pair of monomials. Generated as pairs of words up to renaming variables and swapping sides, there are 3,284 semigroup laws of order at most five, of which 653 have order at most four, Le Floch's number. In commutative form, each side sorted and the variables renamed to give the smallest pair, they become 246 laws, 2, 4, 12, 28, 66 and 134 of orders 0 to 5; a law whose two sides become the same monomial, such as commutativity, becomes $x = x$. Laws equivalent among semigroups remain equivalent among commutative semigroups, so of the 112 laws of order at most four one is taken for each class of laws that are equivalent among semigroups, by the result of Section 7, or equal in commutative form: 24 laws. The 134 laws of order five are all taken. With associativity and commutativity, Eq4512 and Eq43, as axioms, this gives the 160 laws of `equations_csemigroup5.json`, written by `semigroup.py commutative-inputs` (Section 10). The exploration runs without duality, since reversing the words leaves a commutative law unchanged.

Background. The implications of the exploration of Section 7 hold in all semigroups, and so in the commutative ones. Mapped onto the 24 laws, they give 152 background implications, which have 26 closed sets on these laws: the 26 of the 456 semigroup theories that contain commutativity. Since that theory is complete, it contains every implication between these laws that holds in all semigroups, among them those of the ETP graph and of the previous sections. The laws of order five get no background.

Objects. The seeds are the 399 commutative semigroups of at most five elements, up to isomorphism, enumerated by Mace4; 33 monomial semigroups (Section 8.2); the semigroups of the context of Section 7; and the ETP's finite magmas. Of these 1,863 tables, 444 are commutative semigroups, and 15 of them are not redundant.

Expert. The Vampire expert ran with the settings of Section 7, which the preset `configs/csemigroup5.toml` holds:

```
python explorations/equational_theories/explore.py \
    --config explorations/equational_theories/configs/csemigroup5.toml
```

8.2 Countermodels too large to search for

A first exploration, without the monomial semigroups among the seeds, was stopped after 95 questions in 65 minutes. Six of them had been accepted as unconfirmed, each after the full per-law model search of about ten minutes. Their premises were $x^2 = m$ for the six monomials m of degree five that contain a square, from $x^2 = y^2zwu$ to $x^2 = y^5$, and in each the laws left unproved were $x^2 = xyzwu$ and $x^2 = yzwuv$. These premises hold wherever every square, and every monomial containing a square, is one and the same element, and the square-free monomials over five generators refute them all: their 31 products of distinct generators together

with a zero, to which every product with a repeated generator is sent. Every square is 0 there, and so is every monomial containing a square, so that the premises hold, while $xyzwu$, with five distinct generators for the variables, is the product of all five, not 0, and so is $yzwuv$.

No smaller commutative semigroup refutes them. If every square is 0 and a product $a_1 \cdots a_5$ is not, every product of a subset of the a_i divides it and is not 0 either, and the products of two different subsets P and Q differ: if, say, $a \in P \setminus Q$, the product of P times the product of the a_i outside Q contains a twice and is 0, while the product of Q times the same is $a_1 \cdots a_5$. With the zero, this gives at least 32 elements, beyond what the model searches found in ten minutes.

Such semigroups can be constructed instead. For a finite set D of nonzero exponent vectors over n generators that is closed downwards, the monomials of D together with a zero, a product being sent to the zero when it leaves D, form a commutative semigroup: the Rees quotient of the free commutative semigroup by the ideal of the monomials outside D. The seeds include those of at most 32 elements whose monomials are square-free, or have every exponent below e, or have degree at most d, or are square-free of degree at most d, on at most six generators: 33 semigroups, written by `semigroup.py commutative-inputs`. Four of them are among the 15 seeds kept: $x^4 = 0$ on one generator (4 elements), exponents below 3 on three generators (27), the square-free monomials on five (32), and degree at most 2 on six (28).

These semigroups satisfy many laws vacuously: every product of three elements is 0 in the last of them, so that a law with at least three letters on each side holds, and evaluating such a law with seven variables on all 28^7 assignments would take hours. The shortcut for magmas with a zero (Section 3.2) decides them without evaluation, and the 33 semigroups are evaluated on the 160 laws in 18 seconds.

8.3 Results

Laws explored (24 of order at most four, 134 of order five, and the two axioms)	160
Background implications from the semigroup exploration	152
Seeds kept (of 444 commutative semigroups)	15
Questions	165
proved	163
refuted by a finite model found for the question	1
refuted by a free magma	1
unconfirmed or rejected	0
Canonical basis relative to the background	163
relative to no background	192
Closed sets of the theory	45
Classes of equivalent laws	40
Rounds	1
Running time	70 s

Table 7: The exploration of the commutative semigroup laws of order at most five. The finite counterexample, of 12 elements, was found by Vampire's model builder. Of the 40 classes, 24 contain laws of order at most four and 16 only laws of order five; associativity and commutativity are in the class of $x = x$. With every question decided in one round and no symmetry, the canonical basis relative to the background consists of the 163 accepted implications. The running time is the wall-clock time on an 8-core laptop, the evaluation of the seeds included.

Every question was decided (Table 7): the 163 accepted implications are proved, and the context's 17 objects are the 15 seeds, the 12-element counterexample and a free magma. The commutative semigroup laws of order at most five thus generate 45 theories, against 26 for those of order at most four, and fall into 40 classes.

The free magma. The free magma refutes question 71, whose premise is $x^3 = yzwu$, and its row, complete on the 160 laws, comes from the provers' saturation (Section 3.8). It is that of a finite semigroup as well. In a commutative semigroup satisfying $x^3 = yzwu$, all products of four elements are equal, to an element 0 that absorbs every other and equals every cube, so that the free one on seven generators consists of 0 and the monomials of degree at most three with every exponent below 3, 113 elements: the monomial semigroup of that set. The laws explored have at most seven variables, and such a law holds in the free semigroup of the variety on infinitely many generators exactly when it holds in the one on seven. Evaluated on the 160 laws, the 113-element semigroup has exactly the free magma's row, so that every non-implication of this exploration rests on a checked finite semigroup.

Every theory has a model. The 17 objects of the context have distinct intents, and these are the 17 meet-irreducible closed sets. As in Section 7, commutative semigroups are closed under products, so that every closed set is the intent of a commutative semigroup, a finite one, the free magma being replaced by the semigroup of 113 elements, and the implications found are the whole propositional theory of commutative semigroups over these laws.

9 Case study: semigroup laws of order at most five

Le Floch [LF26b, Section 1] writes that equations of order up to five "seem within reach" and that a preliminary investigation finds 380 classes of equivalent laws up to that order. This section determines the classes and all implications between conjunctions of these laws, by sorting the laws into classes and then exploring one law of each, with the result of Section 7 as background. Several questions had counterexamples far beyond the model searches, and two ways of constructing them took the place of the search: truncated free semigroups, built by the expert for each such question, and semigroups built in advance as seeds.

9.1 Classes of laws

Generated as pairs of words up to renaming and swapping sides, there are 3,284 semigroup laws of order at most five (1,796 up to reversal), 653 of them of order at most four. `semigroup.py classes` evaluates each law on 294 semigroups: the 218 of at most four elements up to isomorphism, enumerated by Mace4, and those of the context of Section 7 with their duals. Laws with different values on one of them are not equivalent, which leaves 263 groups of laws that no semigroup separates. Within each, Vampire tries to prove each law equivalent to the smallest one not yet in a class, 10 seconds for each direction, and the laws it proves equivalent form a class; this is repeated with the laws left. In 37 minutes this gave 380 classes, and on the laws of order at most four exactly Le Floch's 122. For 320 pairs of classes Vampire neither proved nor refuted the equivalence of their first laws, so that the proofs alone only bound the number of classes by 380: some classes could have been one. The exploration below shows that they are not, its 380 classes being separated by checked finite semigroups.

9.2 Setup

Laws and background. One law of each class is taken, closed under reversal: a law and its reversal for a class and its dual class, and for each of the 84 self-dual classes a law that is its own reversal where there is one; for the 7 others, both a law and its reversal, given as equivalent in the background. With associativity, these are the 388 laws of `equations_semigroup5.json`, written by `semigroup.py inputs`. The background is the result of Section 7, its implications and their duals mapped onto the laws taken from their classes: 591 implications, which give exactly the 456 theories of Section 7 on the laws of order at most four.

Seeds and expert. The seeds are the 294 semigroups above, the 33 monomial semigroups of Section 8.2, and the Heisenberg groups over $\mathbb{Z}_3$ and $\mathbb{Z}_5$ (Section 9.4): 329 semigroups, of which 61 are not redundant. The expert has the settings of Section 7, with truncated free semigroups of at most 20,000 words (preset `configs/semigroup5.toml`).

9.3 Truncated free semigroups

A first exploration, without truncated free semigroups and without the constructed seeds, asked 278 questions in an hour, four of them accepted as unconfirmed after the full model search. Their premises contain $x^3 = x^4$, $x^3 = yx^3$ and one of $x^2y = yxy^2$, $x^2y = yxyx$, $xyxy = yxx$ and $x^2yx = xy^2$, and the laws left are of the form $x^3 = y^2zx$ or $x^3 = xyz^2$. On two of them, with or without the laws already proved among the premises, neither Vampire in 10 minutes for each law nor E and Twee in 30 minutes or more found a proof, and Mace4 found no model in 15 minutes. Nor do the Rees quotients of the free semigroup refute them, as the monomial semigroups did for commutative semigroups: in such a quotient, the words of a finite set closed under factors together with a zero, a law holds only if every substitution that sends its sides to different words sends both out of the set, so that $x^2y = yxy^2$, with y for x and zx for y, makes y^2zx zero, as y^2zx and $zxyzxzx$ differ.

The countermodels are quotients of a different kind. For laws P and k generators, let S_n be the free semigroup on k generators of the variety of P with every product of n or more letters made zero: the words of length below n together with a zero, modulo the congruence generated by the instances of P, a side of length n or more being the zero. It is computed by closing the instances under multiplication by the generators, a finite computation. S_n satisfies P and is generated by the k generators, so it is a quotient of the free semigroup of the variety, and a law with k variables that fails in S_n at the generators does not follow from P. The elements that divide its failing side, together with the zero, then form a smaller countermodel, the Rees quotient by the ideal of the others. For the four questions:

Third premise law	Open laws	S_n	Countermodel
$x^2y = yxy^2$	$x^3 = y^2zx$	S_8, 271 elements	39
$x^2y = yxyx$	$x^3 = x^2yz$, y^2xz, y^2zx	S_7, 208	36
$xyxy = yxx$	$x^3 = xyz^2$, yxz^2, yzx^2	S_7, 208	36
$x^2yx = xy^2$	$x^3 = xyz^2$, yxz^2, yzx^2	S_8, 271	39

Each took a fraction of a second, and each countermodel was checked against its premise and law. In S_7, y^2zx is zero at the generators, being equal to words of seven letters or more, and in S_8 it is not.

The Vampire expert now tries these semigroups first in its per-law model search, among semigroups (option `--truncation-words`): for each law of the conclusion with at most four variables, S_n for increasing n while it has at most the given number of words, within 30 seconds in all. The instances of a law whose two sides have very different lengths, such as $x = yzwuvt$, are many even when the long side is zero, and the time limit keeps such premises from holding up the exploration.

9.4 Constructed seeds

An exploration with truncated free semigroups, but with the 294 seeds only, asked 764 questions in 98 minutes and accepted five implications as unconfirmed. For four of them, the countermodels had to be constructed by hand. Three, with premises such as $x^2 = y^2zwu$ among many laws and the open law $x^2 = yzwux$, are refuted by the square-free monomials over five generators, of 32 elements (Section 8.2), a commutative semigroup that satisfies all of their premises. The fourth, $x = y^5x \to xyx = yxx \wedge xyz = yxz$, is refuted by a group: in a group, $x = y^5x$ says

that the group has exponent dividing 5, and $xyx = yxx$ that it is commutative. The Heisenberg group over $\mathbb{Z}_5$, of the upper unitriangular 3×3 matrices, is a non-abelian group of exponent 5, with 125 elements, the fewest such a group has. Both are cheap to evaluate on the laws: the monomial semigroup by the shortcut for magmas with a zero (Section 3.2), and the group because a law of at most seven letters holds in a group of exponent 5 only if each variable occurs as often on both sides modulo 5, which leaves at most three variables, while the laws that fail do so on a random assignment. With the 33 monomial semigroups and the Heisenberg groups over $\mathbb{Z}_3$ and $\mathbb{Z}_5$ among the seeds, none of the four remained open.

9.5 Results

Laws explored (one of each of the 380 classes, 7 reversals and associativity)	388
Background implications from the semigroup exploration	591
Seeds kept (of 329)	61
Questions	757
proved	665
refuted by a truncated free semigroup	65
refuted by a model in the pool	19
refuted by a model found by Vampire or Mace4	7
unconfirmed, proved afterwards	1
Canonical basis relative to the background and duality	666
relative to no background, up to duality	825
Classes of equivalent laws	380
Closed sets of the theory	4,450
Rounds	1
Running time	61 min

Table 8: The exploration of the semigroup laws of order at most five. The 91 counterexamples are semigroups of 5 to 56 elements: 83 truncated free semigroups, of which 65 answered the question they were built for and 18 later questions from the pool, 5 models found by Vampire's model builder and 3 by Mace4. The canonical basis, which includes the implication proved afterwards and its dual, has 1,226 implications in all relative to the background and its duals, and 1,478 relative to no background. The running time is the wall-clock time on an 8-core laptop.

Every question but one was decided (Table 8): the 665 accepted implications are proved, and the 91 counterexamples are finite semigroups, checked against every law. The 380 classes are those of the class computation, and they are separated by these semigroups, so that Le Floch's preliminary count of 380 classes is confirmed. The one undecided question has the premise $x^3 = x^4$, $x^3 = x^2yx$, $x^3 = xy^2x$, $x^3 = yx^3$, $x^3 = y^2x^2$, $x^3 = y^3x$, $x^2y = xyxy$, $x^2y = yxy^2$ and the open law $x^3 = y^2zx$, that of the first row of the table above with a larger premise; truncated free semigroups up to S_{11}, of 535 elements, do not separate it. It holds: afterwards, Twee and Prover9 each proved it in about 2 seconds, and Vampire in 16 seconds on four cores, where the expert had given Vampire 3 seconds on all cores and then 10 on one, and E 2 seconds; the proofs are saved with the results (`question324_proofs.txt`), and its dual follows by symmetry. The semigroup laws of order at most five thus generate exactly 4,450 theories, every implication between conjunctions of them being proved and every non-implication witnessed by a checked finite semigroup.

9.6 Certificates

So far the result rests on the provers, on the library's evaluation of laws, and on the bookkeeping of the exploration. It can also be checked from saved evidence, by code that shares nothing with

the exploration and calls no prover. A *certificate* of an implication from laws P to a law $s = t$ is a chain of terms from s to t, each obtained from the last by replacing one subterm with an instance of a law of P, in either direction, or of a lemma. A lemma is an equation proved earlier in the certificate by a chain of the same kind; its chain leaves its variables uninstantiated, so it holds for all their values. The checker, `certificates.py`, accepts a certificate only if every step is such a replacement.

`certify.py` builds the certificates from Vampire's proofs. These name only the inference rule and the parent clauses of each step, so the substitution of each superposition and demodulation is found again by unification, and each derived equation becomes a lemma, with one replacement for each occurrence that Vampire rewrites at once; the constants Vampire introduces for subterms are expanded, and its Skolem constants become the variables of the conclusion. A conclusion of several laws gives a certificate for each: AVATAR splits its negation into a branch for each law, and without AVATAR a law's literal is removed from the negated conclusion when it is refuted. Vampire's inequality splitting, which turns $s \neq t$ into $\neg P(s)$ and $P(t)$ for a new predicate P, is undone: the chains from s and from t end where the two arguments meet. Proofs that the reconstruction cannot follow are redone without clause splitting. The implications to certify are taken from `implications.json`, background and duals included, one law of a conclusion at a time, the laws fewer objects satisfy first, and each only if the implications taken before do not entail it: 4,815 implications. With the 5,826 certificates that each of the 3,284 laws and the law taken from its class imply each other given associativity, this gives 10,641 certificates with 419,120 steps, 13 for the median certificate and at most 977, found in 4 minutes with 8 Vampire processes of 10 seconds each, a proof that fails being retried on all cores for 60 seconds. Compressed, they take 1.7 MB (`certificates.jsonl.xz` in the results). Explorations now save the expert's proofs (Section 10), and `certify.py` starts from them: a proof of an implication also certifies the implications with larger premises, and its mirror image, the factors of every product swapped, the dual implication. Only what they leave is proved again.

`verify.py` then checks, in 7 minutes on 8 cores:

1. that each of the 240 tables of the context is a semigroup and that its row is right, every law being evaluated again by a separate evaluator;
2. every certificate, of which 4,843 are of implications between the 388 laws explored;
3. that the sets closed under these implications, together with associativity and $x = x$ concluded from the empty premise, are exactly the intents of the context: 4,450 sets;
4. that the 3,284 semigroup laws of order at most five, enumerated anew, are each in exactly one class, and certified equivalent to the law taken from their class.

The third check compares two closure systems on the laws. Each object intent is checked to respect every implication, so the closure of a set under the implications is contained in the intersection of the object intents containing it. NextClosure is run with the latter closure, and at each of the 626,451 sets whose closure it computes, the closure under the implications must be the same; it can stop as soon as it reaches the intent, the inclusion making it no larger. What NextClosure does depends only on the closures it is given, so with the implications it would have listed the same sets. The comparison is in the library (`conceptual_exploration.verification`).

Together, the checks establish the result of this section without the provers: every certified implication holds in all semigroups, every intent of the context is that of a semigroup, and since the closed sets coincide, every implication between conjunctions of the laws that holds in the 240 semigroups, and so every one that holds in all semigroups, follows from the certified implications, and every other fails in one of the 240. The 388 columns of the checked context are 380 distinct ones, so the 380 classes are distinct. The background of order four, certified with the rest, is checked too, as is the implication of question 324.

9.7 Lean

The result is also proved in Lean, in the form in which the ETP states its own results, and against the ETP's library, which the Lean project (`lean_order5/`) requires at a fixed commit. `lean_etp.py` writes the Lean files from the results and the certificates:

- The laws are the ETP's `EquationN`. The 2,630 laws of order five used that the ETP does not declare are declared with its `equation` command and numbered by its scheme, which extends past its 4,694 laws of order at most four.
- Each certificate is a theorem in the form of the ETP's, such as

```
theorem Equation3_4512_implies_Equation8 (G : Type*) [Magma G]
  (h3 :  Equation3 G) (h4512 :  Equation4512 G) : Equation8 G,
```

  with the laws of the premise that its chain uses as hypotheses. A lemma becomes a `have`, and a step a `conv` that goes to the subterm and rewrites it with the instance, written out. Certificates of the same implication from the same laws give one theorem: 10,613 theorems, of which the 4,172 with a single hypothesis are `@[equational_result]`s, as the ETP's attribute accepts only those.
- Each of the 240 semigroups is a `finOpTable`, with an `@[equational_result]` theorem `∃ (G : Type) (_ :  Magma G) (_:  Finite G), Facts G [...]  [...]`, as in the ETP's tables of size at most four. The refuted laws are all those of the exploration that it refutes, 76,361 in all. The satisfied ones are, as in the ETP, a few from which the others of its row follow by the implications proved, associativity among them, chosen to be cheap to decide.

A satisfied law is decided by the ETP's `decideFin!`, which evaluates it in the kernel at all values of its variables. Above about 10^6 evaluations the kernel slows out of proportion, so a law that needs more than 10^5 is decided in parts, with its first variables fixed, and the parts are put together by cases on their values: 1,495 parts, for 15 laws in 13 semigroups. A refuted law is decided by `decideFin!` at a counterexample, which `lean_etp.py` finds; searching for the first one would take $3 \cdot 10^8$ evaluations in all.

The build takes about 12 hours of CPU time, three quarters of it for the parts. Every theorem is accepted. The implications depend on no axioms, and the `Facts` on `propext`, `Classical.choice` and `Quot.sound`, the ones the ETP accepts. Lean checks the proofs; `verify.py --lean` checks that their statements are the result, reading the declarations of the laws, the ETP's included, the hypotheses and conclusions of the theorems, and the `Facts`:

1. each law is declared once;
2. the `Facts` of each semigroup include the axioms, and the implications stated give its row from them: the closure of its satisfied laws is disjoint from its refuted ones and together with them covers all laws;
3. the sets closed under the implications stated are exactly the intents of the semigroups;
4. and, with `--classes`, as for the certificates.

It does so in 2 minutes, without reading the Lean proofs. The theory of Section 9.6 thus holds in Lean, on the ETP's definitions, with $x = x$, which no `Facts` lists, as the only law not stated.

10 User's guide

10.1 Installation

The library needs Python 3.11 or later. In a clone of the repository:

```
pip install -e ".[examples]"     # the library, Z3 and Jupyter
```

The Vampire expert additionally needs the `vampire` executable on the path, for example via Homebrew (`brew install vampire`) or a release from `https://vprover.github.io`. The explorations described here used Vampire 5.1.0. The optional Mace4 search needs the `mace4` executable, which comes with Prover9 (`brew install prover9`).

10.2 Describing the laws

Write the laws to a JSON file as in Section 3. Every entry needs an `equation` string; `name` and `id` are optional, and laws without an id are identified by their name or their text in the results. Terms use variables (single letters or identifiers), `*` and parentheses; an unparenthesized product associates to the left, so `x * y * z` is read as $(x * y) * z$. The set must be closed under duality: for every law, a law equal to its dual up to renaming variables and swapping sides must be in the file, or the program stops with an error naming the missing dual. Derived background implications (Section 3.4) require all laws to have the same single variable; background from the ETP (Section 3.5) does not. To explore ETP laws chosen by number, let `etp.py` write the file:

```
python explorations/equational_theories/etp.py equations.txt 3 43 4512 \
    -o my_laws.json
```

where `equations.txt` is the ETP's list of laws; the duals of the chosen laws are added. With `--max-order K --graph graph.json` instead of numbers, it writes one law of each class of equivalent ETP laws of order at most K, closed under duality (Section 6), each with the members of its class. With `--semigroup` in addition, it chooses among the right-associated laws, the ETP's forms of laws on semigroups: the first law of each class, the reversals of these laws, so that the laws are closed under duality among semigroups, and associativity; the file `equations_semigroup.json` lists the 248 laws so chosen for order at most four, to be explored with `--axiom 4512` (preset `configs/semigroup.toml`).

Beyond the ETP's order four, `semigroup.py` generates semigroup laws as pairs of words. Its command `classes` sorts them into classes of laws equivalent among semigroups, and `inputs` writes the laws, the background and the seeds of the exploration of Section 9 from the classes (preset `configs/semigroup5.toml`):

```
python explorations/equational_theories/semigroup.py classes \
    explorations/equational_theories/results/semigroup_order5_classes \
    --max-order 5 --etp-equations etp/equations.txt --seeds \
    explorations/equational_theories/results/\
semigroup_vampire_mace4_e_etp_classes/magmas.json
python explorations/equational_theories/semigroup.py inputs \
    explorations/equational_theories/results/\
semigroup_order5_classes/classes.json --order4 \
    explorations/equational_theories/results/\
semigroup_vampire_mace4_e_etp_classes/implications.json
```

Its command `commutative-inputs` writes the laws, the background and the seeds of the exploration of Section 8 (preset `configs/csemigroup5.toml`):

```
python explorations/equational_theories/semigroup.py commutative-inputs \
    --order4-equations explorations/equational_theories/equations_semigroup.json \
    --order4 explorations/equational_theories/results/\
semigroup_vampire_mace4_e_etp_classes/implications.json \
    --etp-equations etp/equations.txt
```

10.3 Running an exploration

From the root of the repository:

```
python explorations/equational_theories/explore.py \
    --equations my_laws.json --expert vampire
```

Table 9 lists the options. They can also be read from a TOML file with `--config FILE`, keyed by option name without the dashes (`eprover-time-limit = 2`), the options given on the command line overriding the file. Paths are relative to the directory `explore.py` is run from, and patterns among the seed files, such as `etp/magmas/*.txt`, are expanded. The file `explorations/equational_theories/configs/order3.toml` holds the settings for the laws of order at most three, with the ETP data fetched into `etp` (Section 3.5). Every run saves all its options as `settings.toml` in its results directory, so that `--config` with that file runs it again.

10.4 Choosing the expert and the time limits

- Use the **Vampire expert** when the result should be correct. Its confirmed implications are proved, and its counterexamples are checked magmas or free magmas. If it accepts something as unconfirmed or rejects an implication without a counterexample, the summary says so.
- Use the **completion expert** for one-variable laws: it decides every question it can complete, in minutes rather than hours, and certifies its counterexamples.
- Use the **Z3 expert** for a quick first impression, or when Vampire is not available. Treat its accepted implications as conjectures: as the case studies show, they may be false.
- Longer **proving time limits** help when implications are true but hard to prove. The laws of the conclusion in a free magma's row get the same limit, and a law Vampire cannot decide in time stays unknown.
- A short **row time limit**, such as 1 second, speeds up explorations with many laws, where most of a free magma's row is outside the conclusion. The price is more unknown entries, so a free magma refutes fewer later questions by itself.
- Longer **model time limits** help when counterexamples are large. The per-law search usually finds a model faster than the search on the whole conclusion. Adding Mace4 with `--mace4-time-limit` costs up to that many seconds for every question that reaches the per-law search, which is worth it when Vampire's model builder misses counterexamples of moderate size, such as the magmas of size 16 that refuted several unconfirmed implications between laws of order at most three.
- Vampire runs in parallel on all cores when it computes a free magma's row or searches for models law by law, so other work on the machine slows the exploration down.

10.5 Background knowledge

For one-variable laws, background implications usually pay off: they are computed in seconds and remove the questions whose answer is a derivation. Start with `--background-order` equal to the largest order of the laws and a transient order of about twice that; the 31-law exploration used 4 and 10. A larger order finds more implications at a cost that grows quickly. A premise size of 2 is usually enough; compare with `--background-premise-size 0` on a small set of laws if in doubt.

For laws the ETP lists, pass its implication graph and its list of laws:

```
python explorations/equational_theories/explore.py \
    --equations my_laws.json --expert vampire \
    --etp-graph graph.json --etp-equations equations.txt
```

This background costs nothing to compute and leaves only questions whose premise has several laws. It helps most when the implications between pairs of laws are hard to settle. Add `--etp-refutations` to put the graph's refutations into the context as well (Section 3.5); with the ETP's finite magmas seeded, only refutations that they do not account for are kept, such as those by infinite magmas.

Implications known from elsewhere, such as the result of an earlier exploration, can be given with `--background FILE`, in the format of `implications.json`, laws given by their id or else their name or text; every implication of the file must hold. The exploration of Section 8 takes the theory of the semigroup exploration this way.

10.6 Reusing magmas

To start a new exploration with the magmas of an earlier one, pass its `magmas.json` to `--seed-context` (fewer questions) or to `--seed-magmas` (the same questions, answered faster). Free magmas are not reused. The ETP's files of tables can be passed in the same way. Seeded into the context of the exploration of Section 6, the 1374 distinct tables of `All4x4Tables` and `smallest_magma_examples.txt` were reduced to 233 magmas.

10.7 Reading the output

The program prints every question with its answer and writes the same text to `exploration.log`:

```
Number of implications: 111
Number of objects: 1358
Question 685: Eq1: x = x, Eq1629: ... -> Eq8: x = (x * (x * x)), ...
Counterexample found: Free magma over {Eq1, Eq1629, Eq1832, ...}
Satisfies: ...
Does not satisfy: ...
```

The counts are those before the answer; the objects include the duals. An accepted question is followed by `Confirmed` or `Unconfirmed`, and an undecided one by `Rejected` or `Unconfirmed` together with the laws it proved. At the end, the program prints a summary (questions, accepted implications, counterexamples, rejected and conflicting implications, rounds, and the Vampire expert's answers by kind), the simplified implications, any rejected, conflicting and refuted unconfirmed implications, and a few counterexamples.

The results directory contains:

`settings.toml` every option of the run, with the commits of the library and of the ETP data, for `--config`;

`exploration.log` everything printed;

`implications.txt`, `implications.json` every implication found, together with its source, which is one of `background`, `confirmed`, `unconfirmed`, `mapped` and `mapped unconfirmed`; then the refuted unconfirmed implications, the rejected implications, and the conflicting ones with the rejected implication each contradicts;

`simplified_implications.txt` everything found, simplified, each implication with its provenance: the background, simplified the same way, and the implications printed at the

end, each followed by its dual (`mapped background`, `mapped` or `mapped unconfirmed`, from the implication it is the image of) unless it is its own; together they entail every implication found. Each law is written out by its number and text, or by its two words when associativity is assumed;

`magmas.json` the finite magmas of the context with their tables, under the names `M`*k* and `M`*k*`^op`;

`free_magmas.json` the free magmas `F`*k*, each with its premise and its known positive and negative laws;

`witnesses.json` the witnesses `W`*k* of implications with one attribute in their conclusion that the expert showed false without a counterexample, each with the implication it refutes and its row;

`etp_refutations.json` with `--etp-refutations`, the objects `R`*k* standing for refutations of the ETP graph, each with the implication it refutes and its known positive and negative laws;

`context.cxt` the objects with complete rows, in Burmeister's format;

`partial_context.cxt` all objects in the same format, with `?` for unknown entries;

`proofs.jsonl.xz` with the Vampire expert, the proofs it found, one JSON object a line: the premise, in the order of the problem's axioms, the conclusion, the prover, and its proof as printed, Vampire's in TSTP, E's proof object and Prover9's proof (Section 9.6).

The program saves the results only at the end. If it is interrupted, only the log remains.

10.8 Using the library from Python

The runner is a thin layer over the library, which can be used directly:

```
from conceptual_exploration import explore_in_rounds, report_every, simplified_implications
from explorations.equational_theories.magma import ETP
from explorations.equational_theories.vampire import VampireExpert
from explorations.equational_theories.background import background_implications

laws = ETP.load_equations("my_laws.json")
expert = VampireExpert(attributes=laws, prove_time_limit=10,
                       quick_model_time_limit=1, model_time_limit=10)
result = explore_in_rounds(
    laws, expert,
    background_implications=background_implications(laws, 4, 2, 10),
    mappings=[ETP.get_duality_mapping(laws)],
    evaluate_all=True,
    on_question=report_every(100),
)
for implication in simplified_implications(result.base):
    print(implication)
print(result.base.open_questions, expert.answers)
```

`run_magma_exploration` in `explore.py` does the same with the options above and saves the results; the notebook `explore.ipynb` shows smaller examples. Another expert can be plugged in by subclassing `Expert` and implementing `validate`, which returns a refuting `PartialObject`, `None` to accept, or an `Undecided` value.

10.9 Practical remarks

- **Size.** The number of questions grows with the number of closed sets of the final theory, which may be large: 108,635 for 31 laws without a single accepted implication. Expect hours for such an exploration and minutes for one with a few dozen closed sets per law.
- **Long runs.** Keep the terminal open. Putting the machine to sleep is harmless in principle, but a Vampire run in progress at that moment may time out on waking, since the time limits are wall-clock times; this can at worst turn a refutation into an undecided answer, which the summary reports.
- **Reproducibility.** Equations are sorted before they are given to the solvers, so a Z3 exploration is reproducible. Vampire's time limits make a Vampire exploration depend on the machine's speed: a different run may find different counterexamples and ask a different number of questions, but the theory is the same unless a question is left undecided.
- **Machine load.** For the same reason, other work on the machine can turn a refutation into an unconfirmed answer: in an order-3 exploration run alongside other provers, a counterexample of size 15 that Vampire had found in three earlier explorations was missed, and the false implication was refuted only 22 questions later. Run explorations on an otherwise idle machine.
- **Trust.** Finite counterexamples are checked by the library. Proofs and saturation-based CounterSatisfiable verdicts are Vampire's; independent confirmation requires checking Vampire's proofs, as `lean_proofs.py` does for the implications of order at most two, or the free magmas' negative entries, as `completion.py` does when Knuth–Bendix completion of their premises succeeds, or certificates, which `certify.py` builds and `verify.py` checks (Section 9.6), and from which `lean_etp.py` writes Lean proofs in the ETP's form (Section 9.7).

References

[BN98] Franz Baader and Tobias Nipkow. *Term Rewriting and All That.* Cambridge University Press, 1998.

[ETP25] Equational Theories Project contributors. *The Equational Theories Project: Advancing collaborative mathematical research at scale.* arXiv:2512.07087, 2025. `https://arxiv.org/abs/2512.07087`

[GO16] Bernhard Ganter and Sergei Obiedkov. *Conceptual Exploration.* Springer, 2016.

[LF26a] Bruno Le Floch. *Implication semilattice of 990 quasigroup equational laws.* arXiv:2603.29909, 2026. `https://arxiv.org/abs/2603.29909`

[LF26b] Bruno Le Floch. *Lattice of 456 semigroup varieties from equations of order up to 4.* arXiv:2609.39872, 2026. `https://arxiv.org/abs/2609.39872`

Option	Default	Meaning
`--equations PATH`	`equations.json`	the laws to explore
`--expert z3|vampire|completion`	`z3`	which expert answers the questions
`--axiom LAW...`		laws every object satisfies, such as associativity
`--no-duality`	off	do not use the duality symmetry
`--vampire-prove-time-limit S`	10	seconds for a proof, and for each law of the conclusion in a free magma's row
`--vampire-first-time-limit S`	prove limit	seconds for the first proof attempt, on all cores; 3 suffices
`--vampire-row-time-limit S`	prove limit	seconds for each other law of a free magma's row; 0 leaves them unknown
`--vampire-quick-model-time-limit S`	1	seconds for the first finite model search
`--vampire-model-time-limit S`	10	seconds for each per-law finite model search
`--mace4-time-limit S`	0	seconds for Mace4's size sweep in the per-law search, in all; 0 leaves Mace4 out
`--model-premise full|simplified|both`	`full`	the premise the per-law model searches use
`--truncation-words W`	0	with associativity assumed, try truncated free semigroups of at most W words before the per-law model search; 0 leaves them out
`--prover9-time-limit S`	0	seconds for Prover9 on each law of the conclusion that E leaves, before Vampire; 0 leaves Prover9 out
`--eprover-time-limit S`	0	seconds for E first on each law of the conclusion in a free magma's row, Vampire then getting the rest; 0 leaves E out, 2 suffices
`--min-search-size N`	1	smallest table size Z3 tries
`--max-search-size N`	6	largest table size Z3 tries
`--z3-timeout-ms MS`	30000	Z3 time per size; 0 means unbounded
`--background-order N`	off	derive background implications through equations of order at most N (at least the largest order of a law)
`--background-transient-order M`	N	let single steps reach order M
`--background-premise-size K`	2	largest premise of a background implication; 0 computes the canonical basis
`--etp-graph PATH`		the ETP's implication graph, as background; needs `--etp-equations`
`--etp-equations PATH`		the ETP's list of laws, to match the explored laws
`--etp-refutations`	off	add the graph's refutations to the context; needs `--etp-graph`
`--ask-unconfirmed-duals`	off	ask about the duals of unconfirmed implications instead of adding them
`--seed-magmas PATH...`		add magmas to the expert's pool
`--seed-context PATH...`		add magmas to the context before starting
`--background PATH...`		add the implications of files in the format of `implications.json` to the background
`--report-every N`	1	print every Nth question; 0 prints none
`--results-dir PATH`	new directory	where to save the results
`--config FILE`		read options from a TOML file, the command line overriding it

Table 9: Options of `explore.py`. The Z3 options apply to the Z3 expert only, and the Vampire options to the Vampire expert only.